

Table of Contents

App.jsx	2
index.css	3
main.tsx	4
pages\homePage.jsx	5
pages\hooks\UserInfo.js	7
pages\login.jsx	9
pages\protectedRoutes.jsx	14
pages\routerPrint.jsx	15
pages\search.jsx	16
pages\signUp.jsx	24
pages\signUpModule.jsx	32
pages\views\FreelancerView.jsx	38
pages\views>LoadingScreen.jsx	42
pages\views\UserView.jsx	43
pages\views\ViewModule.jsx	44
RouteTracker.jsx	56
socket\MsgBuilder.js	57
socket\MsgParser.js	58
socket\MsgTypes.js	59
socket\RequestHandler.js	61
socket\ResponseHandlers.js	64
socket\SocketContext.jsx	68
store\appointmentsSlice.js	69
store\authSlice.js	70
store\store.js	71

```
import './App.css';
import { Routes, Route, useNavigate } from "react-router-dom";
import { useEffect } from "react";
import LogIn from './pages/login.jsx';
import HomePage from './pages/homePage.jsx';
import ProtectedRoutes from './pages/protectedRoutes.jsx';
import SignUpPage from './pages/signUp.jsx';
import Search from './pages/search.jsx';
import RouteTracker from './RouteTracker.jsx';
import { useSelector } from 'react-redux';

export default function App() {
  const navigate = useNavigate()
  const isLoggedIn = useSelector((state) => state.auth.loggedIn)

  useEffect(() => {
    const lastRoute = localStorage.getItem("lastRoute");

    const isGuestPage = lastRoute === "/login" || lastRoute === "/signup";

    if (isLoggedIn && lastRoute && !isGuestPage) {
      navigate(lastRoute, { replace: true });
    }

    else if (isLoggedIn && (isGuestPage || !lastRoute)) {
      navigate("/", { replace: true });
    }
  }, [navigate, isLoggedIn]);

  return (
    <>
      <RouteTracker />
      <Routes>
        <Route path="/login" element={<LogIn />} />
        <Route path="/signup" element={<SignUpPage />} />
        <Route element={<ProtectedRoutes />>
          <Route path="/" element={<HomePage isPublic={false}/>} />
          <Route path="/profile/:id" element={<HomePage isPublic={true} />} />
          <Route path="/search" element={<Search/>} />
        </Route>
      </Routes>
    </>
  );
}
```

```
@import "tailwindcss";

* {
  scrollbar-width: thin;
  scrollbar-color: #475569 #1e293b;
}
```

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { Provider } from 'react-redux'
import store, { persistor } from './store/store.js'
import { BrowserRouter } from 'react-router-dom'
import { SocketProvider } from './socket/SocketContext.jsx'
import { PersistGate } from 'redux-persist/integration/react';
import App from './App.jsx'
import './index.css'
```

```
const rootElement = document.getElementById('root')!
createRoot(rootElement).render(
  <StrictMode>
    <Provider store={store}>
      <PersistGate loading={null} persistor={persistor}>
        <SocketProvider>
          <BrowserRouter>
            <App />
          </BrowserRouter>
        </SocketProvider>
      </PersistGate>
    </Provider>
  </StrictMode>
)
```

```

import { useSelector, useDispatch } from "react-redux";
import { useNavigate, useParams, useLocation } from "react-router-dom";
import { useState, useEffect } from "react";
import { useSocket } from "../socket/SocketContext";

import LoadingScreen from "../views/LoadingScreen";
import Navbar from "../routerPrint";
import FreelancerView from "../views/FreelancerView";
import UserView from "../views/UserView";
import useUserSync from "../hooks/UserInfo";

export default function HomePage({isPublic=false}) {
  const ws = useSocket();
  const dispatch = useDispatch();
  const navigate = useNavigate();
  const location = useLocation();

  const authUser = useSelector((state) => state.auth.userInfo);
  const token = useSelector((state) => state.auth.userToken);

  const { id } = useParams();

  const [targetId, setTargetId] = useState(id || location.state?.freelancerId || null)

  const [appointmentTimes, setAppointmentTimes] = useState(null)

  const [viewedFreelancer, setViewedFreelancer] = useState(null)

  useUserSync(ws, token, dispatch, navigate, setViewedFreelancer, setAppointmentTimes, targetId, isPublic || (targetId !== null));

  const isLoading = authUser && Object.keys(authUser).length > 0;

  useEffect(() =>{
    const currentUserId = id || location.state?.freelancerId || null
    if (currentUserId == null){
      setViewedFreelancer(null)
      setAppointmentTimes(null)
      return
    }
  }, [id, location.state])

  if (!isLoading) return <LoadingScreen />;

  const renderContent = () => {
    if (authUser.role === "User" && viewedFreelancer) {
      return (
        <FreelancerView
          user={viewedFreelancer}
          isPublic={true}
          appointmentTimes = {appointmentTimes}
        />
      );
    }

    if (authUser.role === "User") {
      return (
        <UserView/>
      )
    }
  }

```

```
    });  
  }  
  
  return <FreelancerView user={authUser} isPublic={false} />;  
};  
  
return (  
  <div className="flex fixed inset-0 bg-slate-900 w-full min-h-screen">  
    <Navbar role={authUser.role}/>  
  
    <div className="ml-60 pt-8 pl-4 w-full overflow-y-auto">  
      <div className="max-w-7xl mx-auto px-4">  
        {renderContent()}  
      </div>  
    </div>  
  </div>  
>);  
}
```

```
// hooks/useUserSync.js
import { useEffect } from "react";
import { UserInfoRequest, MarkReadNotificationsRequest, GetPublicProfileInfoRequest } from "../socket/RequestHandler";
import WebSocketParser from "../socket/MsgParser";
import { handleUserInfoResponse, handleProfileInfoResponse, handleUpdatedAppointmentsResponse, handleMarkedReadNotifications, handleAppointmentTimes, handleAppointmentMadeResponse } from "../socket/ResponseHandlers";
import { logout } from "../store/authSlice";
import { MessageTypes, StatusMessage } from "../socket/MsgTypes";

export default function useUserSync(ws, token, dispatch, navigate, setViewedFreelancer, setAppointmentTimes, target_id, isPublic=false) {
  useEffect(() => {

    if (!ws || !token) return;
    const sendRequest = () =>{ if (!isPublic) {UserInfoRequest(ws, token);} if (target_id) { GetPublicProfileInfoRequest(ws, target_id, token)}}

    const handleOpen = () => sendRequest()
    if (ws.readyState === WebSocket.OPEN) {
      sendRequest();
    } else {
      ws.addEventListener("open", handleOpen);
    }

    const handleMessage = (event) => {

      let info = WebSocketParser(event.data);
      console.log("Recvd - ", info)
      if (MessageTypes.GET_USER_INFO === info.type){
        const [status, msg] = handleUserInfoResponse(dispatch, info);
        if (!status) {
          dispatch(logout());
          navigate("/login");
        }
        else{
          const userId = info.data.user_id;
          MarkReadNotificationsRequest(ws, userId, token)
        }
      }

      else if (MessageTypes.GET_PUBLIC_PROFILE_INFO === info.type){
        const [status, data] = handleProfileInfoResponse(info)
        if (status){
          setViewedFreelancer(data)
        }
        else{
          alert(data)
          navigate("/search")
        }
      }

      else if(MessageTypes.UPDATE_APPOINTMENTS_STATUS === info.type){
        const [status, data] = handleUpdatedAppointmentsResponse(info)
        if (status){
          UserInfoRequest(ws, token)
        }
        else{
          console.log("Error occured ", data)
          UserInfoRequest(ws, token)
        }
      }
    }
  })
}
```

```

    }
  }

  else if(MessageTypes.MARK_READ_NOTIFICATION === info.type){
    const [status, data] = handleMarkedReadNotifications(info)
    if (!status){
      console.log("Didn't manage to mark notifications as read - ", data)
    }
  }

  else if(MessageTypes.GET_APPOINTMENT_TIMES === info.type){
    const [status, msg] = handleAppointmentTimes(info)
    if (status){
      setAppointmentTimes(msg)
      console.log(msg)
    }
    else
      setAppointmentTimes(null)
  }

  else if (MessageTypes.MAKE_APPOINTMENT === info.type) {
    const [status, msg] = handleAppointmentMadeResponse(info);
    if (status) {
      setAppointmentTimes(null)
      alert("Success!");
      navigate("/search");
    } else {
      alert("Error: " + msg);
    }
  }

  else if (MessageTypes.BROAD === info.type){
    if (StatusMessage.TOKEN_BAD in info.data){
      alert("Your session has ended, login again to get service")
      dispatch(logout())
      navigate('/login')
    }
  }
};

ws.addEventListener("message", handleMessage);

return () => {
  ws.removeEventListener("open", handleOpen);
  ws.removeEventListener("message", handleMessage);
};
}, [ws, token, target_id, isPublic]);
}

```



```

//#region imports
import {Link, useNavigate} from 'react-router-dom'
import {useState, useEffect} from 'react'
import {LoginRequest, ForgotPasswordVerificationCodeRequest, ForgotPasswordAuthenticationRequest, ChangePasswordRequest} from '../socket/RequestHandler'
import {useSocket} from '../socket/SocketContext'
import { handleLoginResponse, handleChangePasswordResponse, handleForgotPasswordVerificationCodeResponse, handleForgotPasswordVerifyCodeResponse } from '../socket/ResponseHandlers'
import { useDispatch } from "react-redux"
import WebSocketParser from '../socket/MsgParser'
import {InputField, ErrorMessage, EmailVerification} from './signupModule'
import { MessageTypes } from '../socket/MsgTypes'
//#endregion

function ForgotPassword({goBack, ws, email, setEmail, errorMessage}){

  const [attempted, setAttempted] = useState(false)

  return(
    <div className="fixed inset-0 flex items-center justify-center bg-slate-900 w-full min-h-screen">
      <div className="bg-slate-700 p-10 rounded-xl shadow-md w-96 text-center">
        <h2 className="text-2xl font-bold text-gray-300 mb-4">
          Forgot Password
        </h2>
        <p className="text-gray-300 text-sm mb-6">
          Enter your email and we will send a verification code so you can reset your password.
        </p>
        <form className="flex flex-col items-center space-y-4" onSubmit={(e)=>{e.preventDefault();setAttempted(true); ForgotPasswordVerificationCodeRequest(ws, email);}}>
          >
            <div className="w-full">
              <InputField type="email" name="email" value={email} onChange={(e)=>setEmail(e.target.value)} placeholder="Email"/>
            </div>

            {errorMessage && <ErrorMessage message={errorMessage}/>}
            <button
              type="submit"
              className="w-full p-2 bg-blue-500 text-white rounded-md hover:bg-blue-600 transition"
            >
              Send Reset Email
            </button>
          </form>

          <button
            type = "button"
            className="w-full p-2 mt-4 bg-blue-500 text-white rounded-md hover:bg-blue-600 transition"
            onClick={goBack}
          >
            Back to login
          </button>

        </div>
      </div>
    )
  }

```

```

function ChangePassword({changePasswordRequest, password, setPassword }) {
  const [confirmPassword, setConfirmPassword] = useState("");
  const [errorMessage, setErrorMessage] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();

    if (password !== confirmPassword) {
      setErrorMessage("Passwords do not match");
      return;
    }
    else
      changePasswordRequest();
  };

  return (
    <div className="fixed inset-0 flex items-center justify-center bg-slate-900 w-full min-h-screen">
      <div className="bg-slate-700 p-10 rounded-xl shadow-md w-96 text-center">
        <h2 className="text-2xl font-bold text-gray-300 mb-4">
          Change Password
        </h2>
        <p className="text-gray-300 text-sm mb-6">
          Enter your new password and confirm it.
        </p>

        <form className="flex flex-col items-center space-y-4" onSubmit={handleSubmit}>
          <div className="w-full">
            <InputField
              type="password"
              name="password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              placeholder="New Password"
            />
          </div>

          <div className="w-full">
            <InputField type="password" name="confirmPassword" value={confirmPassword} onChange={(e) => setConfirmPassword(e.target.value)} placeholder="Confirm Password"/>
          </div>

          {errorMessage && <ErrorMessage message={errorMessage} />}

          <button
            type="submit"
            className="w-full p-2 bg-blue-500 text-white rounded-md hover:bg-blue-600 transition"
          >
            Change Password
          </button>
        </form>
      </div>
    </div>
  );
}

export default function LogIn(payload){

```

```

const ws = useSocket();
const navigate = useNavigate()
const dispatch = useDispatch()

const [username, setUsername] = useState("");
const [password, setPassword] = useState("");

const [forgotPassword, setForgotPassword] = useState(false)

const [verificationCode, setVerificationCode] = useState("")

const [email, setEmail] = useState("")

const [serverError, setServerError] = useState("")

const [showVerification, setShowVerification] = useState(false)

const [showChangePass, setShowChangePass] = useState(false)

const [pass, setPass] = useState("")

const handleVerificationCodeSubmit = () =>
{
  ForgotPasswordAuthenticationRequest(ws, email, verificationCode)
}

const handleChangePassword = () =>{
  ChangePasswordRequest(ws, email, pass);
}

useEffect(() =>{
  if (!ws) return;
  const handleMessage = (event) => {
    setServerError("")
    console.log(`Recvd - ${event.data}`)
    const info = websocketParser(event.data)
    console.log(`type - ${info.type}\ndata - ${info.data}`)
    if (info.type == MessageTypes.LOGIN){
      const [loggedIn, message] = handleLoginResponse(info, dispatch);
      if (!loggedIn){
        setServerError(message)
      }
    }

    else if(info.type == MessageTypes.FORGOT_PASSWORD_REQUEST){
      const [veri_sent, message] = handleForgotPasswordVerificationCodeResponse(info)

      if (veri_sent){setShowVerification(true); setForgotPassword(false);
      }else setServerError(message)}

    else if(info.type == MessageTypes.FORGOT_PASSWORD_AUTHENTICATION){
      const [veri_sent,message] = handleForgotPasswordVerifyCodeResponse(info)
      if (veri_sent){setShowVerification(false); setShowChangePass(true)}
      else setServerError(message)
    }

    else if (info.type ==MessageTypes.CHANGE_PASS){
      const [pass_accepted, message] = handleChangePasswordResponse(info)
      if (pass_accepted){
        setForgotPassword(false)
        setShowChangePass(false)
      }
      else setServerError(message)
    }
  }
}, [ws])

```

```

    }
  }
  ws.addEventListener("message", handleMessage);

  return () =>{
    ws.removeEventListener("message", handleMessage)
  }
}, [ws, dispatch, navigate])

if (forgotPassword){
  return <ForgotPassword errorMessage={serverError} goBack={() => setForgotPassword(false)} ws={ws} email={email} setEmail={(email) => setEmail(email)} />
}

if (showVerification)
  return <EmailVerification verificationCode={verificationCode} setVerificationCode={setVerificationCode} handleVerificationCodeSubmit={handleVerificationCodeSubmit} serverError={serverError} />

if (showChangePass)
  return <ChangePassword changePasswordRequest={handleChangePassword} password={password} setPassword={setPass} />
  return(
    <div className="fixed inset-0 flex items-center justify-center bg-slate-900 w-full min-h-screen">
      <div className="bg-slate-700 p-10 rounded-xl shadow-md w-96 text-center">
        <h2 className="text-2xl font-bold text-gray-300 mb-4">Login</h2>

        <form className="flex flex-col items-center space-y-4" onSubmit={(e) => {
          e.preventDefault();
          LoginRequest(username, password, ws);
        }}>
          <div className='w-full'>
            <InputField type={"email"} name={"email"} value={username} onChange={(e) => setUsername(e.target.value)} placeholder={"Email"} />
          </div>

          <div className='w-full'>
            <InputField type={"password"} name={"password"} value={password} onChange={(e) => setPassword(e.target.value)} placeholder={"Password"} />
          </div>

          {serverError && (
            <ErrorMessage message={serverError} />
          )}

          <button
            type="submit"
            className="w-full p-2 bg-blue-500 text-white rounded-md hover:bg-blue-600 transition"
            >
            Login
          </button>

        </form>

        <Link to="/signup" className="block mt-3 text-blue-600 hover:underline">
          Sign up
        </Link>

        <button

```

```
        type="button"
        onClick={() => setForgotPassword(true)}
        className="mt-3 text-blue-600 hover:underline text-sm"
      >
        Forgot Password?
      </button>
    </div>
  </div>
});
```

```
import { useSelector } from "react-redux";
import { Navigate, Outlet } from "react-router-dom";

export default function ProtectedRoutes() {
  const isLoggedIn = useSelector((state) => state.auth.loggedIn);

  if (!isLoggedIn) {
    return <Navigate to="/login" replace />;
  }

  return <Outlet />;
}
```

```

import { Link, useNavigate } from "react-router-dom";
import { useDispatch } from "react-redux";
import { logout } from "../store/authSlice";

function Navbar({ role }) {
  const dispatch = useDispatch();
  const navigate = useNavigate();

  const handleLogout = () => {
    dispatch(logout());
    localStorage.clear();
    navigate("/login");
  };

  return (
    <nav className="fixed left-0 top-0 h-full w-60 bg-slate-800 flex flex-col py-8 px-4 shadow-
lg">

      <div className="flex flex-col gap-2">
        <Link
          to="/"
          className="text-gray-200 px-4 py-2 rounded-md hover:bg-slate-700 transition flex item
s-center gap-3 justify-start"
        >
          <span>■</span> <span>Home Page</span>
        </Link>

        {role === "User" && (
          <Link
            to="/search"
            className="text-gray-200 px-4 py-2 rounded-md hover:bg-slate-700 transition flex i
tems-center gap-3 justify-start"
          >
            <span>■</span> <span>Find Freelancers</span>
          </Link>
        )}
      </div>

      <div className="mt-auto border-t border-slate-700 pt-4">
        <button
          onClick={handleLogout}
          className="w-full flex items-center gap-3 justify-start text-red-500 font-semibold px
-4 py-2 rounded-md hover:bg-red-950/30 transition duration-200"
        >
          <span className="text-xl">■</span>
          <span>Logout</span>
        </button>
      </div>

    </nav>
  );
}

export default Navbar;

```

```

import { useState, useEffect, useRef } from "react";
import { useSelector, useDispatch } from "react-redux";
import Navbar from "../routerPrint";
import { MapPin, Briefcase, Search, ChevronRight, Ellipse, User, Star, CreditCard, ArrowLeft }
from "lucide-react";
import { GetAllJobs, GetCitiesByJob, GetMinimalFreelancerInfo, GetPublicProfileInfoRequest } fr
om "../socket/RequestHandler";
import { useSocket } from "../socket/SocketContext";
import websocketParser from "../socket/MsgParser";
import { MessageType, StatusMessage } from "../socket/MsgTypes";
import { handleJobsResponse, handleCitiesResponse, handleMinimalFreelancerInfoResponse } from ".
../socket/ResponseHandlers"
import LoadingScreen from "../views/LoadingScreen";
import { useNavigate } from "react-router-dom";
import { logout } from "../store/authSlice";

function SearchBar({ ws, token, handleSubmit }) {
  const navigate = useNavigate()
  const authUser = useSelector((state) => state.auth.userInfo);
  const dispatch = useDispatch()

  const [allJobTypes, setAllJobTypes] = useState([]);
  const [jobCitiesFromServer, setJobCitiesFromServer] = useState([]);

  const [searchJob, setSearchJob] = useState("");
  const [selectedJob, setSelectedJob] = useState(null);
  const [isJobOpen, setIsJobOpen] = useState(false);

  const [searchCity, setSearchCity] = useState("");
  const [selectedCity, setSelectedCity] = useState(null);
  const [isCityOpen, setIsCityOpen] = useState(false);

  const jobRef = useRef(null);
  const cityRef = useRef(null);

  useEffect(() => {
    const handleClickOutside = (event) => {
      if (jobRef.current && !jobRef.current.contains(event.target)) setIsJobOpen(false);
      if (cityRef.current && !cityRef.current.contains(event.target)) setIsCityOpen(fals
e);
    };
    document.addEventListener("mousedown", handleClickOutside);
    return () => document.removeEventListener("mousedown", handleClickOutside);
  }, []);

  useEffect(() => {
    if (!ws) return;
    const handleMessage = (event) => {
      let info = websocketParser(event.data);

      if (MessageType.BROAD == info.type){
        if (StatusMessage.TOKEN_BAD in info.data){
          alert("Your session has ended, login again to get service")
          dispatch(logout())
          navigate('/login')
        }
      }
      else if (info.type == MessageType.GET_JOBS) {
        const [status, message] = handleJobsResponse(info);
        if (status) setAllJobTypes(message);
      }
    };
  }, [ws]);

```



```

    } else if (info.type == MessageTypes.GET_CITIES_BY_JOB) {
      const [status, message] = handleCitiesResponse(info);
      if (status) setJobCitiesFromServer(message);
    }
  };

  const setup = () => {
    ws.addEventListener("message", handleMessage);
    GetAllJobs(ws, token);
  };

  if (ws.readyState == WebSocket.OPEN) setup();
  else ws.addEventListener("open", setup, { once: true });

  return () => {
    ws.removeEventListener("message", handleMessage);
  };
}, [ws, token]);

const resetSearch = () => {
  setSelectedJob(null)
  setSearchJob("")
  setSelectedCity(null)
  setSearchCity("")
  setJobCitiesFromServer([]);
};

const isLocked = selectedJob && selectedCity;
return (
  <div className="fixed inset-0 bg-[#020617] flex overflow-hidden font-sans antialiased text-slate-200">
    <Navbar role={authUser?.role} />

    <div className="absolute inset-0 overflow-hidden pointer-events-none select-none">
      <div className="absolute -top-24 -right-24 w-1/2 aspect-square bg-blue-600/10 blur-[120px] rounded-full" />
      <div className="absolute -bottom-24 -left-24 w-1/3 aspect-square bg-indigo-600/10 blur-[100px] rounded-full" />
    </div>

    <main className="flex-1 ml-60 h-full relative flex flex-col items-center justify-center p-8 lg:p-16 z-10">

      <div className="w-full max-w-5xl space-y-12">

        <header>
          <h1 className="text-7xl md:text-8xl lg:text-9xl font-black text-white tracking-tight leading-[0.9]">
            Hire <span className="text-blue-500">Quality</span><br />
            <span className="opacity-10 italic">Freelancers</span>
          </h1>
        </header>

        <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">

          <div className="relative group" ref={jobRef}>
            <Briefcase className={`absolute left-7 top-1/2 -translate-y-1/2 w-7 h-7 z-20 transition-colors ${selectedJob ? 'text-blue-400' : 'text-slate-500'}` />

            <input
              type="text"
              readOnly={!selectedJob}

```

```

        placeholder={selectedJob ? "" : "Service"}
        className={`w-full bg-slate-900/40 border-2 text-white text-3xl py-
10 pl-20 pr-32 rounded-3xl outline-none transition-all backdrop-blur-md
        ${selectedJob ? 'border-blue-500/40' : 'border-slate-800 focus
:blue-500 hover:border-slate-700'}`}
        value={searchJob}
        onFocus={() => !selectedJob && setIsJobOpen(true)}
        onChange={(e) => setSearchJob(e.target.value)}
    />

    {selectedJob && (
        <button
            onClick={resetSearch}
            className="absolute right-6 top-1/2 -translate-y-1/2 z-30 px-5
py-2 rounded-xl text-xs font-bold tracking-widest bg-blue-500/10 text-blue-400 border border-bl
ue-500/20 hover:bg-blue-500/20 transition-colors uppercase"
            >
                Change
            </button>
        )}

    {isJobOpen && (
        <div className="absolute z-50 w-full mt-3 bg-slate-900 border bord
er-slate-800 rounded-2xl shadow-2xl max-h-80 overflow-y-auto ring-1 ring-white/5">
            {allJobTypes.filter(j => j.toLowerCase().includes(searchJob.to
LowerCase()))}.map(job => (
                <button
                    key={job}
                    onClick={() => { setSelectedJob(job); setSearchJob(job
); setIsJobOpen(false); GetCitiesByJob(ws, job, token); }}
                    className="w-full text-left px-8 py-6 text-xl text-sla
te-300 hover:bg-blue-600 hover:text-white flex justify-between items-center transition-colors g
roup/item"
                    >
                        {job}
                        <ChevronRight className="w-5 h-5 opacity-0 group-hover
/item:opacity-100 transition-opacity" />
                    </button>
                )}
            </div>
        )}
    </div>

    <div className={`relative transition-all duration-500 ${!selectedJob ? 'op
acity-40 grayscale pointer-events-none' : 'opacity-100'}`} ref={cityRef}>
        <MapPin className={`absolute left-7 top-1/2 -translate-y-1/2 w-7 h-7 z
-20 ${selectedCity ? 'text-blue-400' : 'text-slate-500'}`} />
        <input
            type="text"
            readOnly={!selectedCity}
            placeholder={selectedCity ? "" : "In which city?"}
            className={`w-full bg-slate-900/40 border-2 text-white text-3xl py-
10 pl-20 pr-32 rounded-3xl outline-none transition-all backdrop-blur-md
            ${selectedCity ? 'border-blue-500/40' : 'border-slate-800 focu
s:border-blue-500 hover:border-slate-700'}`}
            value={searchCity}
            onFocus={() => !selectedCity && setIsCityOpen(true)}
            onChange={(e) => setSearchCity(e.target.value)}
        />

    {selectedCity && (
        <button

```

```

        onClick={() => { setSelectedCity(null); setSearchCity(""); }}
        className="absolute right-6 top-1/2 -translate-y-1/2 z-30 px-5
py-2 rounded-xl text-xs font-bold tracking-widest bg-blue-500/10 text-blue-400 border border-bl
ue-500/20 hover:bg-blue-500/20 transition-colors uppercase"
      >
        Change
      </button>
    )}

    {isCityOpen && (
      <div className="absolute z-50 w-full mt-3 bg-slate-900 border bord
er-slate-800 rounded-2xl shadow-2xl max-h-80 overflow-y-auto ring-1 ring-white/5">
        {jobCitiesFromServer.filter(c => c.toLowerCase().includes(sear
chCity.toLowerCase()))}.map(city => (
          <button
            key={city}
            onClick={() => { setSelectedCity(city); setSearchCity(
city); setIsCityOpen(false); }}
            className="w-full text-left px-8 py-6 text-xl text-sla
te-300 hover:bg-blue-600 hover:text-white transition-colors"
          >
            {city}
          </button>
        ))}
      </div>
    )}
  </div>
</div>

  <div className={`transition-all duration-700 delay-100 ${isLocked ? 'translate
-y-0 opacity-100' : 'translate-y-8 opacity-0 pointer-events-none'}`}>
    <button
      onClick={() => handleSubmit(selectedCity, selectedJob)}
      className="w-full relative group p-[1px] rounded-3xl overflow-hidden bg
-slate-800 hover:bg-blue-500 transition-colors duration-300"
    >
      <div className="bg-white text-black py-10 rounded-[calc(1.5rem-1px)] fo
nt-black text-4xl flex items-center justify-center gap-6 group-hover:bg-transparent group-hover
:text-white transition-all">
        <Search className="w-10 h-10 stroke-[3px]" />
        <span>SEARCH NOW</span>
      </div>
    </button>

    <p className="mt-6 text-center text-slate-500 text-base">
      Searching for <span className="text-blue-400">{selectedJob}</span> in
<span className="text-blue-400">{selectedCity}</span>
    </p>
  </div>

</div>
</main>
</div>
)
}

function FreelancersMenu({ws, token, goBack, city, job}){
  const dispatch = useDispatch()
  const navigate = useNavigate()

  const authUser = useSelector((state) => state.auth.userInfo)

```

```

const [gotFreelancersInfo, setGotFreelancersInfo] = useState(false)
const [freelancersInfo, setFreelancersInfo] = useState(null)

const hasRequested = useRef(false)

const handleOnClick = (id) =>{
  GetPublicProfileInfoRequest(ws, id, token)
  navigate(`/profile/${id}`, { state: { freelancerId: id } });
}

useEffect(() => {
  const handleMessage = (event) =>{
    if (!ws) return;

    const info = websocketParser(event.data)
    if (info.type == MessageTypes.BROAD){
      if (StatusMessage.TOKEN_BAD in info.data){
        alert("Invalid token, ending session. Log in to keep searching");
        dispatch(logout())
        navigate("/login")
      }
    }

    else if(info.type === MessageTypes.GET_MINIMAL_FREELANCER_INFO){
      const [status, msg] = handleMinimalFreelancerInfoResponse(info)
      if (status){
        console.log(msg)
        setFreelancersInfo(msg)
        setGotFreelancersInfo(true);
      }
      else{
        alert(msg)
        goBack()
      }
    }
  }

  ws.addEventListener("message", handleMessage)

  if (!hasRequested.current){
    GetMinimalFreelancerInfo(ws, token, city, job)
    hasRequested.current = true
  }
  return () => ws.removeEventListener("message", handleMessage)
}, [ws, token, city, job])

if (!gotFreelancersInfo) return <LoadingScreen/>

return (
  <div className="fixed inset-0 bg-[#020617] flex overflow-hidden font-sans antialiased text-slate-200">
    <Navbar role={authUser?.role} />

    <div className="absolute inset-0 overflow-hidden pointer-events-none select-none">
      <div className="absolute top-[-20%] left-[10%] w-[800px] h-[800px] bg-blue-600/5 blur-[120px] rounded-full" />
    </div>

    <main className="flex-1 ml-60 h-full relative flex flex-col p-8 lg:p-16 overflow-y

```

-auto scrollbar-hide">

```

    <div className="w-full max-w-6xl mx-auto z-10">

      <header className="flex flex-col lg:flex-row lg:items-end justify-between
gap-10 mb-20">
        <div className="space-y-6">
          <div className="flex items-center gap-4 text-blue-500 font-bold tr
acking-widest text-xs uppercase">
            <span className="w-8 h-px bg-blue-500/50"></span>
            Available Professionals
          </div>
          <h1 className="text-6xl md:text-7xl font-black text-white tracking-
tighter leading-none">
            {job} <br />
            <span className="text-slate-600">in</span> {city}
          </h1>
        </div>

        <button
          onClick={goBack}
          className="group flex items-center gap-3 bg-slate-900 border borde
r-slate-800 hover:border-blue-500 px-6 py-4 rounded-2xl text-slate-400 hover:text-white transit
ion-all duration-300 shadow-xl"
        >
          <ArrowLeft className="w-5 h-5 group-hover:-translate-x-1 transitio
n-transform" />
          <span className="font-bold text-sm uppercase tracking-widest">Chan
ge Search</span>
        </button>
      </header>

      <div className="grid grid-cols-1 md:grid-cols-2 xl:grid-cols-3 gap-8">
        {Object.entries(freelancersInfo).map(([pro_id, data]) => (
          <div
            key={pro_id}
            className="group relative bg-slate-900/30 border border-slate-8
00 hover:border-blue-600/40 rounded-[2rem] p-8 transition-all duration-500 hover:shadow-[0_20px
_50px_rgba(8,_112,_184,_0.1)] flex flex-col"
          >
            <div className="flex justify-between items-start mb-10">
              <div className="flex items-center gap-2 px-3 py-1.5 bg-bl
ue-500/10 border border-blue-500/20 rounded-lg text-blue-400">
                <Star className="w-3.5 h-3.5 fill-blue-400" />
                <span className="font-black text-xs uppercase tracking
-tighter">
                  {data.rating ? `${data.rating} Rating` : "New"}
                </span>
              </div>
              <div className="text-right">
                <span className="block text-[10px] font-bold text-slat
e-500 uppercase tracking-widest mb-1">Hourly Rate</span>
                <span className="text-3xl font-black text-white tracki
ng-tight">${data.price}</span>
              </div>
            </div>

            <div className="flex-1 space-y-4 mb-12">
              <div className="flex items-center gap-4">
                <div className="w-12 h-12 bg-slate-800 rounded-xl flex
items-center justify-center border border-slate-700 group-hover:scale-110 transition-transform
duration-500">

```

```

        <User className="text-slate-500 group-hover:text-b
    ue-400 w-6 h-6" />
        </div>
        <div>
            <h3 className="font-black text-lg text-white group
    -hover:text-blue-400 transition-colors">Verified Professional</h3>
            <p className="text-[10px] text-slate-500 font-bold
    uppercase tracking-widest leading-none">Pro-Find Member</p>
            </div>
        </div>

        <p className="text-slate-400 leading-relaxed text-base ita
    lic line-clamp-4 pt-2">
            {data.description ? `${data.description}` : "This pr
    ofessional hasn't provided a bio yet, but their credentials meet our high platform standards."}
        </p>
    </div>

    <button
        onClick={() => handleClick(pro_id)}
        className="w-full bg-white text-black py-5 rounded-2xl fon
    t-black text-sm tracking-widest flex items-center justify-center gap-2 group-hover:bg-blue-600
    group-hover:text-white transition-all duration-300 active:scale-95"
    >
        <span>VIEW PROFILE</span>
        <ChevronRight className="w-4 h-4" />
    </button>
</div>
    )}
</div>

    {Object.keys(freelancersInfo).length === 0 && (
        <div className="flex flex-col items-center justify-center py-32 border
    border-dashed border-slate-800 rounded-[3rem] bg-slate-900/10">
            <div className="w-16 h-16 bg-slate-800 rounded-full flex items-cen
    ter justify-center mb-6">
                <Search className="text-slate-500 w-8 h-8" />
            </div>
            <h3 className="text-white text-2xl font-black tracking-tight mb-2">
    No matches found</h3>
            <p className="text-slate-500 text-lg max-w-sm text-center">
                Try broadening your search criteria or checking a neighboring
    city.
            </p>
        </div>
    )}
</div>
</main>
</div>
)
}

```

```

export default function SearchPage({}) {
    const ws = useSocket()
    const token = useSelector((state) => state.auth.userToken)

    const [serching, setSearching] = useState(true)
    const [gotFreelancers, setGotFreelancers] = useState(false)

```

```
const [serchFields, setSearchFields] = useState({city:null, job:null})

const handleSearch = (city, job) =>{
  setSearchFields({city, job})
  setSearching(false)
}

return (
  <>
    {serching ? (
      <SearchBar ws={ws} token={token} handleSubmit={handleSearch} />
    ) : ( <FreelancersMenu ws={ws} token={token} goBack={() => setSearching(true)}
city={serchFields.city} job={serchFields.job} />
    )}
  </>
)
}
```

```

import { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import { BackToLogin, freelancerProfessions, RolePopup, InputField, ErrorMessage, SingleChoice
DropDownMenu, MultiChoiceDropDownMenu, israeliLocalities, EmailVerification } from "../signupModule";
import { useSocket } from "../socket/SocketContext";
import { SignUpRequest, VerificationRequest } from "../socket/RequestHandler";
import websocketParser from "../socket/MsgParser";
import { handleSignUpResponse, handleVerificationResponse } from "../socket/ResponseHandlers";
import { MessageType } from "../socket/MsgTypes";

function UserSignUp({ userInfo, setUserInfo, onSubmit, serverError }) {
  const [confirmPassword, setConfirmPassword] = useState("");
  const [passwordMismatch, setPasswordMismatch] = useState(false);
  const [infoEntered, setInfoEntered] = useState(true);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setUserInfo(prev => ({
      ...prev,
      [name]: value
    }));
  };

  const handleSubmit = (e) => {
    console.log(`User name - ${userInfo.name}\nUser email - ${userInfo.email}\nUser password - ${userInfo.password}`);
    e.preventDefault();
    const hasEmptyFields = Object.values(userInfo).some(value => !value);
    if (!hasEmptyFields && confirmPassword !== "") {
      setInfoEntered(true);

      if (userInfo.password !== confirmPassword) {
        setPasswordMismatch(true);
        setConfirmPassword("");
      }

      else {
        setPasswordMismatch(false);
        onSubmit();
      }
    }

    else {
      setPasswordMismatch(false);
      setInfoEntered(false);
    }
  };

  return (
    <div className="fixed inset-0 flex items-center justify-center bg-slate-900 w-full min-h-screen">
      <BackToLogin />
      <div className="bg-slate-700 p-12 rounded-xl shadow-lg w-120 text-center">
        <h1 className="text-gray-300 text-2xl font-bold mb-4">
          Please enter your information below
        </h1>
        <form className="flex flex-col gap-4" onSubmit={handleSubmit}>
          <InputField name="name" value={userInfo.name} onChange={handleChange} placeholder="Username" />

          <InputField name="email" value={userInfo.email} onChange={handleChange} placeholder="Email" />
        </form>
      </div>
    </div>
  );
}

```



```

r={"Email"} type={"email"}/>

      <InputField name={"password"} value={userInfo.password} onChange={handleChange} placeholder={"password"} type={"password"} />

      <InputField name={"password2"} value={confirmPassword} onChange ={(e) => setConfirmPassword(e.target.value)} placeholder={"Confirm password"} type={"password"} />

      {passwordMismatch && (
        <ErrorMessage message={"Passwords do not match"}/>
      )}

      {!infoEntered && (
        <ErrorMessage message={"Not all fields were filled"}/>
      )}

      {serverError && (
        <ErrorMessage message={serverError}/>
      )}

      <button
        type="submit"
        className="bg-blue-400 hover:bg-blue-500 text-white font-semibold py-2 rounded-lg shadow-md mt-2 transition-all"
      >
        Sign Up
      </button>
    </form>
  </div>
</div>
);
}

```

```

function FreelancerSignUp({ freelancerInfo, setFreeLancerInfo, onSubmit, serverError }) {
  const [confirmPassword, setConfirmPassword] = useState("");
  const [passwordMismatch, setPasswordMismatch] = useState(false);
  const [infoEntered, setInfoEntered] = useState(true);

  const [startHour, setStartHour] = useState("");
  const [startMinute, setStartMinute] = useState("")
  const [endHour, setEndHour] = useState("")
  const [endMinute, setEndMinute] = useState("")

  const [jobDurationHour, setJobDurationHour] = useState("")
  const [jobDurationMinute, setJobDurationMinute] = useState("")

  const [professions, setProfessions] = useState([]);
  const [cities, setCities] = useState([]);

  let widthForBigDropDowns = "480px";
  let widthForSmallDropDowns = "90px";
  let timeWidth = '50px';

  useEffect(() => {
    if (startHour !== "" && startMinute !== "") {
      setFreeLancerInfo(prev => ({
        ...prev,
        startWorking: `${startHour}:${startMinute}`
      }
    )
  )
}

```

```

    }));
  }
}, [startHour, startMinute])

useEffect(() => {
  if (endHour !== "" && endMinute !== ""){
    setFreelancerInfo(prev => ({
      ...prev,
      finishWorking: `${endHour}:${endMinute}`
    }));
  }
}, [endHour, endMinute])

useEffect(() => {
  if (jobDurationHour !== "" && jobDurationMinute !== "")
  {
    setFreelancerInfo(prev => ({
      ...prev, jobDuration: `${jobDurationHour}:${jobDurationMinute}`
    }))
  }
}, [jobDurationHour, jobDurationMinute])

useEffect(() => {
  const loadProfessions = async () => {
    const profs = await freelancerProfessions();
    setProfessions(profs);
  };
  loadProfessions();

  const loadCities = async () => {
    const cityList = await israeliLocalities();
    setCities(cityList);
  };
  loadCities();
}, []);

const handleChange = (e) =>{
  const {name, value} = e.target;
  setFreelancerInfo(prev =>({
    ...prev,
    [name]:value
  }))
}

const handleSubmit = (e) =>{
  console.log(`
User name - ${freelancerInfo.name}
User email - ${freelancerInfo.email}
User password - ${freelancerInfo.password}
Profession - ${freelancerInfo.profession}
Cities - ${freelancerInfo.cities}
Years of expertise - ${freelancerInfo.years}
Working hours - ${freelancerInfo.startWorking}-${freelancerInfo.finishWorking}
Avg job duration - ${freelancerInfo.jobDuration}
Self description - ${freelancerInfo.description }
`)

  e.preventDefault()
  const hasEmptyFields = Object.entries(freelancerInfo).some(([key, value]) => {
    if (Array.isArray(value)) return value.length === 0;
  })
}

```

```

    if (typeof value === "string") return value.trim() === "";
    if (typeof value === "number") return value === 0;
    return !value;
  });

  if (!hasEmptyFields && confirmPassword !== ""){
    setInfoEntered(true)
    if(freelancerInfo.password!==confirmPassword){
      setPasswordMismatch(true)
      setConfirmPassword("");}
    else{
      setPasswordMismatch(false)
      onSubmit()
    }
  }
  else{
    setPasswordMismatch(false)
    setInfoEntered(false)
  }
}

}

return (
  <div className="fixed inset-0 flex items-center justify-center bg-slate-900 w-full min-h-screen">
    <BackToLogin/>
    <div className="bg-slate-600 p-8 rounded-xl shadow-lg w-140 max-h-[90vh] overflow-y-auto text-center scrollbar-hide">
      <h1 className="text-gray-300 text-xl font-bold mb-4">
        Please enter your information below
      </h1>
      <form className="text-gray-300 flex flex-col gap-4" onSubmit={handleSubmit}>
        <InputField name={name} value={freelancerInfo.name} onChange={handleChange} placeholder="Username"/>

        <InputField name="email" value={freelancerInfo.email} onChange={handleChange} placeholder="Email" type="email"/>

        <InputField name="password" value={freelancerInfo.password} onChange={handleChange} placeholder="password" type="password" />

        <InputField name="password2" value={confirmPassword} onChange={e => setConfirmPassword(e.target.value)} placeholder="Confirm password" type="password" />
        <SingleChoiceDropDownMenu
          placeholder="Profession"
          name="profession"
          options={professions}
          value={freelancerInfo.profession}
          onChange={professionPicked => setFreelancerInfo(prev => ({...prev, profession:professionPicked}))}
          customWidth={widthForBigDropDowns}
        />

        <MultiChoiceDropDownMenu
          name="cities"
          options={cities}
          value={freelancerInfo.cities}
          onChange={selectedCities => setFreelancerInfo(prev => ({...prev, cities:selectedCities}))}
          placeholder="Select your areas"
          customWidth={widthForBigDropDowns}
        />
      </form>
    </div>
  </div>
)

```

```

    />

    <SingleChoiceDropDownMenu
      name={"years"}
      options={Array.from({length:41}, (_, i) =>i)}
      value ={freelancerInfo.years}
      onChange={ (yearsPicked) => setFreelancerInfo(prev => ({...prev, years:yearsPicked}))}

      placeholder="Select years of expertise"
      customWidth={widthForBigDropDowns}
    />

    <div className="flex items-center gap-2">
      <span className="text-m font-bold w-50">Start time</span>
      <SingleChoiceDropDownMenu
        name={"startTime"}
        options={Array.from({length:24}, (_,i)=>String(i).padStart(2,"0"))}
        value={startHour}
        onChange={(startHourPicked) => setStartHour(startHourPicked)}
        customWidth={widthForSmallDropDowns}
        placeholder=""
      />

      <span className="text-2xl font-bold">:</span>

      <SingleChoiceDropDownMenu
        name={"startMinute"}
        options={Array.from({length:60}, (_,i)=>String(i).padStart(2,"0"))}
        value={startMinute}
        onChange={(startMinutePicked) => setStartMinute(startMinutePicked)}
        customWidth={widthForSmallDropDowns}
        placeholder=""
      />
    </div>

    <div className="flex items-center gap-2">
      <span className="text-m font-bold w-50">End time</span>
      <SingleChoiceDropDownMenu
        name={"endTime"}
        options={Array.from({length:24}, (_,i)=>String(i).padStart(2,"0"))}
        value={endHour}
        onChange={(endHourPicked) => setEndHour(endHourPicked)}
        customWidth={widthForSmallDropDowns}
        placeholder=""
      />

      <span className="text-2xl font-bold">:</span>

      <SingleChoiceDropDownMenu
        name={"endMinute"}
        options={Array.from({length:60}, (_,i)=>String(i).padStart(2,"0"))}
        value={endMinute}
        onChange={(endMinutePicked) => setEndMinute(endMinutePicked)}
        customWidth={widthForSmallDropDowns}
        placeholder=""
      />
    </div>

    <div className="flex items-center gap-2">

```

```

    <span className="text-m font-bold w-50">Job duration time</span>
    <SingleChoiceDropDownMenu
      name={"jobStart"}
      options={Array.from({length:24}, (_,i)=>String(i).padStart(2, "0"))}
      value={jobDurationHour}
      onChange={(jobDurationHourPicked) => setJobDurationHour(jobDurationHourPicked)}
    >
      customWidth={widthForSmallDropDowns}
      placeholder=""
    </>

    <span className="text-2xl font-bold">:</span>

    <SingleChoiceDropDownMenu
      name={"jobEnd"}
      options={Array.from({length:60}, (_,i)=>String(i).padStart(2, "0"))}
      value={jobDurationMinute}
      onChange={(jobDurationMinutePicked) => setJobDurationMinute(jobDurationMinuteP
icked)}}
      customWidth={widthForSmallDropDowns}
      placeholder=""
    </>
  </div>

  <InputField name={"description"} value={freelancerInfo.description} onChange={hand
leChange} placeholder={"Description"}/>

  <SingleChoiceDropDownMenu
    name={"hourPrice"}
    options={Array.from({length:1001}, (_,i) => i)}
    value={freelancerInfo.hourPrice}
    onChange={(pricePicked) => setFreeLancerInfo(prev => ({
      ...prev,
      hourPrice: pricePicked
    })))}
    placeholder={"Price per hour $$$$"}
    customWidth={widthForBigDropDowns}/>

  {passwordMismatch && (
    <ErrorMessage message={"Passwords do not match"}/>
  )}

  {!infoEntered && (
    <ErrorMessage message={"Not all fields were entered"}/>
  )}

  {serverError && (
    <ErrorMessage message={serverError}/>
  )}

  <button
    type="submit"
    className="bg-blue-400 hover:bg-blue-500 text-white font-semibold py-2 rounded-
lg shadow-md mt-2 transition-all"
  >
    Sign Up
  </button>
</form>
</div>
</div>

```

```

    );
}

export default function SignUpPage() {
  const ws = useSocket()
  const navigate = useNavigate();

  const [role, setRole] = useState(null);
  const [userInfo, setUserInfo] = useState({ name:"", email:"", password:""});
  const [freelancerInfo, setFreelancerInfo] = useState({
    name:"",
    email:"",
    password:"",
    profession:"",
    cities: [],
    years: 0,
    startWorking: "",
    finishWorking:"",
    jobDuration:0,
    description: "",
    hourPrice:""
  });

  const [serverError, setServerError] = useState("");

  const [showVerification, setShowVerification] = useState(false);
  const [verificationCode, setVerificationCode] = useState("");

  const handleSignUpPageSubmit = () => {
    setServerError("")
    console.log(role)
    if (role === "User")
      SignUpRequest(ws, userInfo, role);
    else if (role === "Freelancer")
      SignUpRequest(ws, freelancerInfo, role);
  }

  const handleVerificationCodeSubmit = () =>{
    setServerError("");
    if (role==="User")
      VerificationRequest(ws, userInfo.email, verificationCode, role);
    else if (role === "Freelancer")
      VerificationRequest(ws, freelancerInfo.email, verificationCode, role);
  }

  useEffect(() =>{
    if (!ws) return;
    const handleMessage = (event) =>{
      console.log(`Recvd - ${event.data}`)
      const info = websocketParser(event.data)

      if (info.type === MessageTypes.VERIFICATION){
        const [infoGood, serverMessage] = handleVerificationReponse(info);
        if (infoGood){
          navigate("/login");
        }
      }
    }
  })
}

```

```

    }
    else{
      setServerError(serverMessage)
    }

  }
  else{
    const [infoGood, errorMessage]= handleSignUpResponse(info)

    if (infoGood){
      setServerError("")
      setShowVerification(true);
    }
    else{
      setServerError(errorMessage)
    }
  }
}

ws.addEventListener("message", handleMessage);

return () =>{
  ws.removeEventListener("message", handleMessage);
}
},[ws])

return (
  <>
    { showVerification ?(
      <EmailVerification verificationCode={verificationCode} setVerificationCode={setVerificationCode} handleVerificationCodeSubmit={handleVerificationCodeSubmit} serverError={serverError}/>
    ): !role ? (
      <RolePopup onSelect={ setRole } />
    ) : role === "User" ? (
      <UserSignUp userInfo={userInfo} setUserInfo={setUserInfo} serverError={serverError} onSubmit={handleSignUpPageSubmit}/>
    )
    : (
      <FreelancerSignUp freelancerInfo={freelancerInfo} setFreelancerInfo={setFreelancerInfo} serverError={serverError} onSubmit={handleSignUpPageSubmit}/>
    )}
  </>
)
}

```

```

import {Link} from "react-router-dom"
import Select from "react-select"
import {useRef, useState} from "react"
import { StatusMessage } from "../socket/MsgTypes";
import {useNavigate} from 'react-router-dom'
import {VerificationRequest} from "../socket/RequestHandler"

//#region lists

export const freelancerProfessions = async () => {
  try {
    const response = await fetch("/professional.txt");
    const text = await response.text();
    return text
      .split("\n")
      .map(line => line.trim())
      .filter(Boolean);
  } catch (error) {
    console.error("Error loading professions:", error);
    return [];
  }
};

export const israeliLocalities = async () => {
  try {
    const response = await fetch("/cities.txt");
    const text = await response.text();
    return text
      .split("\n")
      .map(line => line.trim())
      .filter(Boolean);
  } catch (error) {
    console.error("Error loading cities:", error);
    return [];
  }
};
//#endregion

//#region Popups and buttons (HTML code)
export function BackToLogin()
{
  return(
    <Link
      to="/login"
      className="absolute top-4 left-4 bg-slate-900 text-blue-500 px-4 py-2 rounded-lg shadow
hover:bg-slate-600 transition"
    >
      ← Back to Login
    </Link>
  );
}

export function ErrorMessage({message}){
  return (
    <p className="text-red-600 font-bold text-base mt-1">
      {message}
    </p>
  )
}

```



```

export function InputField({ label, type="text", name, value, onChange, placeholder }) {
  return (
    <div className="flex flex-col text-left">
      {label && <label className="mb-1 font-medium">{label}</label>}
      <input
        type={type}
        required
        name={name}
        value={value || ""}
        onChange={onChange}
        placeholder={placeholder}
        className="border text-gray-300 bg-slate-700 border-blue-400 rounded-lg px-4 py-2 focus:outline-none focus:ring-2 focus:ring-blue-400 placeholder-gray-400"
      />
    </div>
  );
}

```

```

export function SingleChoiceDropDownMenu({label, name, options, value, onChange, customWidth,
placeholder = "Select an option"}){
  let orderedOptions;
  try{
    orderedOptions = options.slice().sort((a,b) => a.localeCompare(b))
  } catch(error){
    orderedOptions = options
  }
}

```

```

const formattedOptions = orderedOptions.map(option => ({value:option, label:option}));

```

```

return (
  <div className="flex flex-col text-left">
    {label && <label className="mb-1 font-medium">{label}</label>}
    <Select
      options = {formattedOptions}
      value = {value ? {value:value, label:value}:null}
      onChange = {(selected) => onChange(selected.value)}
      placeholder={placeholder}
      classNamePrefix = 'react-select'
      menuPlacement = "auto"
      maxHeight={200}
      styles={{
        container: (base) => ({ ...base, width: customWidth || '100%' }),
        control: (base, state) => ({
          ...base,
          backgroundColor: '#374151',
          borderColor: '#60a5fa',
          boxShadow: state.isFocused ? '0 0 0 2px #60a5fa' : 'none',
          color: 'ffffff',
          '&:hover': { borderColor: '#3b82f6' },
        }),
        menu: (base) => ({ ...base, backgroundColor: '#374151', color: 'ffffff' }),
        option: (base, state) => ({
          ...base,
          backgroundColor: state.isFocused ? '#1e293b' : '#374151',
          color: 'ffffff',
        }),
        singleValue: (base) => ({ ...base, color: 'ffffff' }),
        placeholder: (base) => ({ ...base, color: '#cbd5e1' }),
        input: (base) => ({ ...base, color: '#d1d5db' })
      }}
    />
  </div>
)

```

```

    />
  </div>
)
}

```

```

export function MultiChoiceDropDownMenu({label, name, options, value, onChange, customWidth, placeholder = "Select options"}){
  const sortedOptions = options.slice().sort((a,b) => (a.localeCompare(b)))

  const orderedOptions = sortedOptions.map(v => ({value:v, label:v}))
  const formattedValue = value?.map(v => ({value:v ,label:v})) || []

  return (
    <div className="flex flex-col text-left">
      {label && <label className="mb-1 font-medium">{label}</label>}
      <Select
        options = {orderedOptions}
        value = {formattedValue}
        onChange ={(selected) => {const selectedValue = selected ? selected.map(s => s.value)
: []; onChange(selectedValue)}}
        placeholder={placeholder}
        classNamePrefix = 'react-select'
        menuPlacement = "auto"
        maxHeight={200}
        isMulti ={true}
        styles={{
          container: (base) => ({ ...base, width: customWidth || '100%' }),
          control: (base, state) => ({
            ...base,
            backgroundColor: '#374151',
            borderColor: '#60a5fa',
            boxShadow: state.isFocused
              ? '0 0 0 2px #60a5fa'
              : 'none',
            color: 'ffffff',
            '&:hover': {
              borderColor: '#3b82f6',
            },
          }),
          menu: (base) => ({
            ...base,
            backgroundColor: '#374151',
            color: 'ffffff',
          }),
          option: (base, state) => ({
            ...base,
            backgroundColor: state.isFocused ? '#1e293b' : '#374151',
            color: 'ffffff',
          }),
          multiValue: (base) => ({
            ...base,
            backgroundColor: '#1e293b',
            color: 'ffffff',
          }),
          multiValueLabel: (base) => ({
            ...base,
            color: 'ffffff',
          }),
          multiValueRemove: (base) => ({
            ...base,
            color: 'ffffff',

```

```

        ':hover': {
            backgroundColor: '#3b82f6',
            color: 'ffffff',
        },
    })),
    singleValue: (base) => ({ ...base, color: 'ffffff' }),
    placeholder: (base) => ({ ...base, color: '#cbd5e1' }),
  })
  />
</div>
)
}
//endregion

//region page components
export function RolePopup({ onSelect }) {
  return (
    <div className="fixed inset-0 flex items-center justify-center bg-slate-900 w-full min-h-screen">
      <BackToLogin/>

      <div className="bg-slate-700 p-8 rounded-xl shadow-lg w-80 text-center">
        <h2 className="text-lg font-semibold text-gray-300 mb-6">
          Hey! Please pick how you would like to be presented in Pro-Find
        </h2>

        <div className="flex justify-between">
          <button
            className="flex-1 bg-blue-400 hover:bg-blue-500 text-white py-2 rounded mr-2 transition"
            onClick={() => onSelect("User")}
          >
            User
          </button>
          <button
            className="flex-1 bg-blue-400 hover:bg-blue-500 text-white py-2 rounded ml-2 transition"
            onClick={() => onSelect("Freelancer")}
          >
            Freelancer
          </button>
        </div>
      </div>
    </div>
  );
}

export function EmailVerification({ verificationCode, setVerificationCode, handleVerificationCodeSubmit, serverError }) {
  const navigate = useNavigate()
  const inputsRef = useRef([])
  const [error, setError] = useState("")

  const handleChange = (value, index) => {
    const codeArray = verificationCode.split("")
    codeArray[index] = value
    const newCode = codeArray.join("")
    setVerificationCode(newCode)

    if (value && index < 5) {
      inputsRef.current[index + 1].focus()
    }
  }
}

```

```

    }
  }

  const handleKeyDown = (e, index) => {
    if (e.key === "Backspace") {
      const codeArray = verificationCode.split("")
      codeArray[index] = ""
      setVerificationCode(codeArray.join(""))

      if (index > 0) {
        inputsRef.current[index - 1].focus()
      }
    }
  }

  const handleSubmit = () => {
    if (verificationCode.length < 6) {
      setError("Bad format: 6 digits required")
      return
    }

    if (!/^\d{6}$/.test(verificationCode)) {
      setError("Bad format: Only numbers allowed")
      return
    }

    setError("")
    handleVerificationCodeSubmit()
  }

  return (
    <div className="fixed inset-0 flex items-center justify-center bg-slate-900 w-full min-h-screen p-4">

      <div className="bg-slate-700 p-10 rounded-xl shadow-lg w-full max-w-md text-center">

        <h2 className="text-4xl font-semibold text-gray-300 mb-2">
          Insert Verification Code
        </h2>

        <p className="text-gray-400 mb-6">
          A verification code was sent to your email.
        </p>

        <div className="flex justify-center gap-4 mb-6">
          {[0,1,2,3,4,5].map((i) => (
            <input
              key={i}
              ref={el => (inputsRef.current[i] = el)}
              type="text"
              maxLength="1"
              value={verificationCode[i] || ""}
              onChange={(e) => handleChange(e.target.value, i)}
              onKeyDown={(e) => handleKeyDown(e, i)}
              className="w-14 h-16 text-center text-white text-3xl border-b-4 border-blue-400 focus:outline-none rounded-sm flex items-center justify-center"
            />
          ))}
        </div>

        {error && <ErrorMessage message={error} />}
      </div>
    </div>
  )

```

```
    {serverError && <ErrorMessage message={serverError}/>}

    <button
      onClick={handleSubmit}
      className="mt-6 bg-blue-400 hover:bg-blue-500 text-white font-semibold py-2 px-6 rounded transition text-xl w-60 h-15"
    >
      Submit
    </button>

  </div>

</div>
)
}
//#endregion
```

```

import { WelcomeBack, WelcomeTo, NotificationsView, UpcomingAppointmentsView, PendingAppointments, FreelancerInfo, AppointmentBooking } from './ViewModule'
import { useSelector } from 'react-redux'
import { useRef, useState, useEffect } from 'react'
import { useSocket } from '../../socket/SocketContext'
import { Calendar } from 'lucide-react'
import { BookAppointment, GetAvailableWorkTimes } from '../../socket/RequestHandler'
import LoadingScreen from './LoadingScreen'
import { ErrorMessage } from './signupModule'

function FreelancerPublic({ user, ws, token, appointmentTimes }) {
  const userId = useSelector((state) => state.auth.userInfo?.id)
  const profId = user.id
  const username = user.username
  const job = user.job
  const cities = user.cities
  const description = user.description
  const years = user.years
  const jobDuration = user.job_duration
  const rating = user.rating
  const price = user.price

  const [makeAppointment, setMakeAppointment] = useState(false)

  const [app, setApp] = useState({ prof_id: profId, user_id: userId })

  useEffect(() => {
    if (Object.keys(app).length === 2) { return; }
    else { BookAppointment(ws, app, token); }
  }, [app])

  useEffect(() => {
    if (makeAppointment) {
      GetAvailableWorkTimes(ws, profId, token)
    }
  }, [makeAppointment])

  const handleConfirm = (newAppData) => {
    const fullAppointment = { ...app, ...newAppData };
    BookAppointment(ws, fullAppointment, token);
  };

  return (<
    { !makeAppointment ? (
      <div className="p-8 max-w-6xl mx-auto animate-fadeIn space-y-8">
        <WelcomeTo username={username}/>

        <FreelancerInfo profession={job} serviceCities={cities} description={
description} years_experience={years} jobDuration={jobDuration} rating={rating} pricePerHour={
price} />

        <button
          onClick={() => setMakeAppointment(true)}
          className="mt-8 w-full py-4 bg-gradient-to-r from-blue-600 to-indigo-600
0 hover:from-blue-500
          hover:to-indigo-500 text-white font-bold rounded-2xl shadow-lg shadow-b
lue-500/20 transition-all
          duration-300 transform hover:-translate-y-1 active:scale-95 flex items-

```

center justify-center gap-3 group"

```

        <Calendar className="w-5 h-5 group-hover:animate-pulse" />
        <span>Make Appointment</span>
      </button>
    </div>
  ) : (appointmentTimes ? (

    <AppointmentBooking
      availability={appointmentTimes}
      jobDuration={jobDuration}
      onConfirm={handleConfirm}
      onCancel={() => setMakeAppointment(false)}
      price={price}/>

    ) : (<LoadingScreen/>)

  )
}
</>
)
}

function FreelancerPrivate({ws, token}){
  const acceptAppointmentsRef = useRef(null);
  const scrollToRef = () =>(acceptAppointmentsRef.current?.scrollIntoView({behavior:"smooth"
}))

  const userInfo = useSelector((state) => state.auth.userInfo)

  const username = useSelector((state) => state.auth.userInfo?.name)
  const notifications = useSelector((state) => state.auth.notifications || [])
  const appointments = useSelector((state) => state.appointment?.freelancerAppointments || [

])

  const confirmedAppointments = appointments.filter((app) => app.status === "Confirmed")
  const pendingAppointments = appointments.filter((app) => app.status === "Requested")

  return (
    <div className="p-8 max-w-6xl mx-auto animate-fadeIn space-y-8">
      <WelcomeBack username={username}/>

      <FreelancerInfo profession={userInfo?.job} serviceCities={userInfo?.cities} descri
ption={userInfo?.description} years_experience={userInfo?.years} jobDuration={userInfo?.job_du
ration} rating={userInfo?.rating} pricePerHour={userInfo?.hour_price} />

      <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
        <div className="bg-slate-800/50 border-l-4 border-purple-500 p-6 rounded-xl sha
dow-lg backdrop-blur-sm">
          <p className="text-slate-400 text-sm font-medium uppercase tracking-wider"
>Notifications</p>
          <p className="text-3xl font-bold text-white mt-1">{notifications.length}</
p>
        </div>

        <div className="bg-slate-800/50 border-l-4 border-blue-500 p-6 rounded-xl shado
w-lg backdrop-blur-sm">
          <p className="text-slate-400 text-sm font-medium uppercase tracking-wider"
>Scheduled Appointments</p>

```

```

        <p className="text-3xl font-bold text-white mt-1">
          {confirmedAppointments.length}
        </p>
      </div>

      <div className="bg-slate-800/50 border-l-4 border-emerald-500 p-6 rounded-xl shadow-lg backdrop-blur-sm">
        <p className="text-slate-400 text-sm font-medium uppercase tracking-wider">
          Pending Appointments</p>
        <p className="text-3xl font-bold text-white mt-1">
          {pendingAppointments.length}
        </p>
      </div>
    </div>

    <div className="bg-gradient-to-br from-blue-600/20 to-purple-600/20 py-16 px-8 rounded-3xl border border-blue-500/20 flex flex-col items-center justify-center shadow-2xl">
      <h3 className="text-2xl font-bold text-white mb-8">Quick Actions</h3>
      <div className="grid grid-cols-1 sm:grid-cols-1 gap-6 w-full max-w-2xl">
        <button onClick={scrollToRef} className="flex items-center justify-center py-10 bg-slate-800 hover:bg-slate-700 text-white rounded-xl transition-all text-lg font-semibold border border-slate-600 shadow-lg hover:scale-105">
          Accept Pending appointments
        </button>
      </div>
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-2 gap-8">
      <NotificationsView notifications={notifications} />

      <UpcomingAppointmentsView appointments={confirmedAppointments}/>
    </div>

    <div ref={acceptAppointmentsRef}>
      <PendingAppointments appointments={pendingAppointments} ws={ws} token={token}/>
    </div>
  </div>

)
}

export default function FreelancerView({ user, isPublic, appointmentTimes }) {
  const ws = useSocket()
  const token = useSelector((state) => state.auth.userToken)

  return (
    <>
      {isPublic ? (
        <FreelancerPublic user={user} ws={ws} token={token} appointmentTimes={appointmentTimes}/>
      ) : (

```



```
        <FreelancerPrivate ws={ws} token={token}/>
      )
    }
  </>
)
}
```

```
import { useState, useEffect } from "react";
import Navbar from "../routerPrint";
export default function LoadingScreen() {
  const [dots, setDots] = useState(".");

  useEffect(() => {
    const interval = setInterval(() => {
      setDots((prev) => (prev === "... " ? "." : prev + "."));
    }, 500);
    return () => clearInterval(interval);
  }, []);

  return (
    <>
      <div className="fixed inset-0 bg-slate-900 flex items-center justify-center">
        <h1 className="text-white text-5xl font-bold">Loading{dots}</h1>
      </div>
    </>
  );
}
```

```

import { useSelector } from "react-redux"
import { Link } from "react-router-dom"
import { WelcomeBack, NotificationsView, UpcomingAppointmentsView } from "../ViewModule"

export default function UserView() {
  const username = useSelector((state) => state.auth.userInfo?.name)
  const notifications = useSelector((state) => state.auth.notifications || [])
  const appointments = useSelector((state) => state.appointment?.userAppointments || [])

  return (
    <div className="p-8 max-w-6xl mx-auto animate-fadeIn space-y-8">
      <WelcomeBack username={username}/>

      <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div className="bg-slate-800/50 border-l-4 border-purple-500 p-6 rounded-xl shadow-lg backdrop-blur-sm">
          <p className="text-slate-400 text-sm font-medium uppercase tracking-wider">
            >Notifications</p>
          <p className="text-3xl font-bold text-white mt-1">{notifications.length}</p>
        </div>
        <div className="bg-slate-800/50 border-l-4 border-blue-500 p-6 rounded-xl shadow-w-lg backdrop-blur-sm">
          <p className="text-slate-400 text-sm font-medium uppercase tracking-wider">
            >Scheduled Appointments</p>
          <p className="text-3xl font-bold text-white mt-1">{appointments.length}</p>
        </div>
      </div>

      <div className="bg-gradient-to-br from-blue-600/20 to-purple-600/20 py-16 px-8 rounded-3xl border border-blue-500/20 flex flex-col items-center justify-center shadow-2xl">
        <h3 className="text-2xl font-bold text-white mb-8">Quick Actions</h3>
        <div className="grid grid-cols-1 sm:grid-cols-1 gap-6 w-full max-w-2xl">
          <Link to="/search" className="flex items-center justify-center py-10 bg-slate-800 hover:bg-slate-700 text-white rounded-xl transition-all text-lg font-semibold border border-slate-600 shadow-lg hover:scale-105">
            Find Freelancers
          </Link>
          {/* <Link to="/schedule" className="flex items-center justify-center py-10 bg-slate-800 hover:bg-slate-700 text-white rounded-xl transition-all text-lg font-semibold border border-slate-600 shadow-lg hover:scale-105">
            View Calendar
          </Link> */}
        </div>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-8">
        <NotificationsView notifications={notifications}/>
        <UpcomingAppointmentsView appointments={appointments}/>
      </div>
    </div>
  )
}

```

```

import { useEffect, useState } from "react";
import { CalendarIcon, CheckCircle, Info, Check, X, Briefcase, MapPin, AlignLeft, Calendar, Star, Clock, FileText, User, DollarSign } from "lucide-react"
import { UpdateAppointmentsStatusRequest } from "../../socket/RequestHandler";
import { ErrorMessage } from "../signupModule";

export function WelcomeBack({ username }){
  return (<
    <header>
      <h1 className="text-4xl font-extrabold text-white tracking-tight">
        Home Page
      </h1>
      <p className="text-lg text-slate-400 mt-2">
        Welcome back <span className="text-blue-400 font-semibold">{username}</spa
n>
      </p>
    </header>
  </>)
}

export function WelcomeTo({ username }){
  return (
    <
      <header>
        <h1 className="text-4xl font-extrabold text-white tracking-tight">
          Freelancer Page
        </h1>
        <p className="text-lg text-slate-400 mt-2">
          Welcome to <span className="text-blue-400 font-semibold">{username}</span>'s pag
e
        </p>
      </header>
    </>
  )
}

export function NotificationsView({ notifications }){
  return (
    <div className="bg-slate-800/40 p-6 rounded-2xl border border-slate-700 flex flex-col h-[500px]">
      <h3 className="text-xl font-semibold text-white mb-4 text-left">Recent Activity</h3>
      <div className="space-y-4 overflow-y-auto pr-2 custom-scrollbar flex-grow">
        {notifications.length > 0 ? (
          notifications.map((note, idx) => (
            <div key={idx} className="flex flex-col items-start text-left text-slate-300 text-sm py-3 border-b border-slate-700/50 last:border-0 hover:bg-slate-700/20 transition-colors rounded-lg px-2">
              <div className="flex items-center justify-between w-full mb-1">
                <div className="flex items-center">
                  {!note.is_read ? (
                    <span className="w-2 h-2 rounded-full bg-blue-500 mr-3"></span>
                    <span className="text-blue-400 font-semibold text-xs uppercase tracking-wider">
                      From: {note.from_name}
                    </span>
                  ) : (
                    <span></span>
                  )
                }
              </div>
            </div>
          )
        ) : (
          <div></div>
        )
      }
    </div>
  )
}

```

```

        </>
      ) : (
        <span className="text-gray-400 font-semibold text-xs upper c
ase tracking-wider">
          From: {note.from_name}
        </span>
      )}
    </div>
    <span className="text-[10px] text-slate-500 font-mono italic">
      {note.created_at}
    </span>
  </div>
  <p className="ml-5 text-slate-200 leading-relaxed text-left">
    {note.message}
  </p>
</div>
))
): {
  <div className="flex flex-col items-start justify-center h-full">
    <p className="text-slate-500 italic text-sm">No new activity found.</p>
  </div>
}
</div>
</div>)
}

export function UpcomingAppointmentsView({ appointments }) {
  return (
    <div className="bg-slate-800/40 p-6 rounded-2xl border border-slate-700 flex flex-col h-[500px]">
      <h3 className="text-xl font-semibold text-white mb-6 text-left">Upcoming Appointments</h3>
      <div className="space-y-4 overflow-y-auto pr-2 custom-scrollbar flex-grow">
        {appointments.length > 0 ? (
          //Here I can control if I want to show a limited amount of appointments
          appointments.map((app, idx) => (
            <div key={idx} className="bg-slate-700/30 p-4 rounded-xl text-slate-300 text-sm border border-slate-600/50 shadow-inner flex flex-col items-start">
              <div className="flex flex-col space-y-2 w-full">
                <p className="flex items-start justify-start">
                  <span className="text-blue-400 font-semibold uppercase text-[10px] tracking-wider w-24 shrink-0 mt-1">Date:</span>
                  <span className="text-white font-medium">{app.display_date}</span>
                </p>
                <p className="flex items-start justify-start">
                  <span className="text-blue-400 font-semibold uppercase text-[10px] tracking-wider w-24 shrink-0 mt-1">Start Time:</span>
                  <span className="text-white font-medium">{app.start_time}</span>
                </p>
                <p className="flex items-start justify-start min-w-0 w-full">
                  <span className="text-blue-400 font-semibold uppercase text-[10px] tracking-wider w-24 shrink-0 mt-1">
                    Description:
                  </span>
                  <span
                    title={app.details}
                    className="text-white font-medium break-words text-left fl

```

```

x-1 min-w-0, truncate"
      >
        {app.details}
      </span>
    </p>
  </div>
</div>
))
): (
  <div className="flex flex-col items-start justify-center h-full">
    <p className="text-slate-500 italic text-sm">No upcoming appointments found
  </p>
  </div>
  </div>
)
</div>
}

```

```

export function PendingAppointments({ appointments = [], ws, token }) {
  const [decisions, setDecisions] = useState({});
  const [selectedApp, setSelectedApp] = useState(null);

  const totalDecisions = Object.keys(decisions).length;
  const gridLayout = "grid grid-cols-[1fr_1.2fr_0.5fr_0.5fr_1fr_2fr_0.4fr_1fr]";

  const [allowed, setAllowed] = useState(true)

  const actualDecisionsCount = Object.values(decisions).filter(status => status !== null).length;

  const handleDecision = (appId, status) => {
    setDecisions((prev) => ({ ...prev, [appId]: status }));
  };

  const handleFinalSubmit = () => {
    UpdateAppointmentsStatusRequest(ws, decisions, token);
    setDecisions({})
  };

  useEffect(() => {
    const seen = new Set();
    let conflictFound = false;

    const acceptedIds = Object.keys(decisions).filter(
      (id) => decisions[id] === "accepted"
    );

    for (const appId of acceptedIds){
      const appData = appointments.find((a) => String(a.id) === String(appId))
      if (appData){
        const slotKey = `${appData.date}-${appData.start_time}`
        if (seen.has(slotKey)){conflictFound=true; break}
        seen.add(slotKey)
      }
    }
    setAllowed(!conflictFound)
  }, [decisions, appointments])

  return (

```

```

<>
  <div className="w-full bg-slate-900 text-white rounded-xl shadow-2xl overflow-hidden border border-slate-800">

    <div className={` ${gridLayout} gap-4 bg-slate-800 p-4 font-bold border-b border-slate-900 text-[10px] uppercase tracking-wider`} >
      <div>Date</div>
      <div>Client</div>
      <div>Start</div>
      <div>End</div>
      <div>Address</div>
      <div>Details</div>
      <div className="text-center">Info</div>
      <div className="text-center">Action</div>
    </div>

    <div className="divide-y divide-slate-800">
      {appointments.map((app) => (
        <div key={app.id} className={` ${gridLayout} gap-4 p-4 items-center transition-all $
{
  decisions[app.id] === 'accepted' ? 'bg-emerald-900/10' :
  decisions[app.id] === 'cancelled' ? 'bg-rose-900/10' : 'hover:bg-slate-800/30'
}`} >
        <div className="text-slate-400 text-xs">{app.display_date}</div>
        <div className="font-semibold truncate text-sm">{app.person_name}</div>
        <div className="text-slate-300 text-xs">{app.start_time}</div>
        <div className="text-slate-300 text-xs">{app.end_time}</div>
        <div className="truncate text-xs text-slate-500" title={app.address}>{app.address}</div>

        <div className="text-sm text-slate-300 truncate" title={app.details}>{app.details}</div>

        <div className="flex justify-center">
          <button
            onClick={() => setSelectedApp(app)}
            className="p-2 text-blue-400 hover:text-blue-300 hover:bg-blue-400/10 rounded
            -lg transition-all"
          >
            <Info size={18} />
          </button>
        </div>

        <div className="flex justify-center gap-2">
          <button
            onClick={() => handleDecision(app.id, 'accepted')}
            className={`p-2 rounded-full transition-all ${decisions[app.id] === 'accepted' ? 'bg-emerald-500 text-white' : 'bg-slate-800 text-slate-500 hover:bg-emerald-600 hover:text-white'}`
          >
            <Check size={16} />
          </button>
          <button
            onClick={() => handleDecision(app.id, 'cancelled')}
            className={`p-2 rounded-full transition-all ${decisions[app.id] === 'cancelled' ? 'bg-rose-500 text-white' : 'bg-slate-800 text-slate-500 hover:bg-rose-600 hover:text-white'}`
          >
            <X size={16} />
          </button>
        </div>
      ))}
    </div>
  </div>

```

```

    </div>

    <div className="p-4 bg-slate-800/80 border-t border-slate-700 flex items-center justify-between">
      <div className="text-sm text-slate-400">
        {totalDecisions > 0 ? (
          <span>You have marked <b className="text-white">{totalDecisions}</b> appointments
        ) : (
          <span>Select appointments to accept or decline.</span>
        )}
      </div>
      {!allowed && (<ErrorMessage message={"Selected two appointments with the same date and starting time"} />)}

      <button
        onClick={handleFinalSubmit}
        disabled={actualDecisionsCount === 0 || !allowed}
        className={`flex items-center gap-2 px-6 py-2.5 rounded-lg font-bold transition-all transform active:scale-95 ${
          actualDecisionsCount > 0 && allowed
            ? 'bg-blue-600 hover:bg-blue-500 shadow-[0_0_15px_rgba(37,99,235,0.4)] text-white'
            : 'bg-slate-700 text-slate-500 cursor-not-allowed'
        }`}
      >
        Confirm & Save Changes
      </button>
    </div>

    </div>
    <AppointmentDetailModal
      appointment={selectedApp}
      onClose={() => setSelectedApp(null)}
    />
  </>
);
}

export function FreelancerInfo({ profession, serviceCities, description, years_experience, jobDuration, rating, pricePerHour }) {
  return (
    <div className="bg-slate-800/40 border border-slate-700 p-8 rounded-3xl shadow-2xl backdrop-blur-md animate-fadeIn">
      <div className="flex flex-col md:flex-row md:items-center justify-between gap-4 mb-8">
        <div>
          <h2 className="text-3xl font-extrabold text-white tracking-tight">{profession}</h2>
          <div className="flex items-center gap-2 mt-2 text-yellow-400">
            <Star className="w-5 h-5 fill-current" />
            <span className="font-bold text-lg">{rating || "New"}</span>
            <span className="text-slate-500 text-sm font-normal">Rating</span>
          </div>
        </div>
        <div className="bg-blue-600/20 border border-blue-500/30 px-5 py-2 rounded-2xl">
          <p className="text-blue-400 text-sm uppercase font-bold tracking-widest">Experience</p>
          <p className="text-white text-xl font-bold">{years_experience} Years</p>
        </div>
      </div>

```



```

    </div>

    {pricePerHour > 0 && (
      <div className="bg-green-600/20 border border-green-500/30 px-5 py-2 rounded-2xl flex items-center gap-3">
        <DollarSign className="w-8 h-8 text-green-400" />
        <div>
          <p className="text-green-400 text-sm uppercase font-bold tracking-wider">Rate</p>
          <p className="text-white text-xl font-bold">${pricePerHour}/hr</p>
        </div>
      </div>
    )}
  </div>

  <hr className="border-slate-700/50 mb-8" />

  <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
    <div className="space-y-4">
      <div className="flex items-center gap-2 text-slate-400">
        <FileText className="w-5 h-5" />
        <h4 className="font-semibold uppercase text-xs tracking-wider">About Me</h4>
      </div>
      <p className="text-slate-300 leading-relaxed italic">
        "{description}"
      </p>
    </div>

    <div className="grid grid-cols-1 gap-4">
      <div className="flex items-start gap-4 p-4 bg-slate-900/40 rounded-xl border border-slate-700/30">
        <MapPin className="w-6 h-6 text-purple-500 mt-1" />
        <div>
          <p className="text-slate-500 text-xs font-bold uppercase">Service Cities</p>
          <p className="text-slate-200">{serviceCities || 'Remote'}</p>
        </div>
      </div>

      <div className="flex items-start gap-4 p-4 bg-slate-900/40 rounded-xl border border-slate-700/30">
        <Clock className="w-6 h-6 text-blue-500 mt-1" />
        <div>
          <p className="text-slate-500 text-xs font-bold uppercase">Avg. Project Duration</p>
          <p className="text-slate-200">{jobDuration}</p>
        </div>
      </div>
    </div>
  </div>
);
}

export function AppointmentDetailModal({ appointment, onClose }) {
  if (!appointment) return null;

  return (

```

```

<div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-[#020617]/90 backdrop-blur-md">
  <div className="bg-[#0f172a] border-l-4 border-blue-500 w-full max-w-lg rounded-br-3xl rounded-tr-xl rounded-bl-xl shadow-[20px_20px_60px_rgba(0,0,0,0.5)] overflow-hidden transform rotate-1 md:rotate-0">
    <div className="pt-8 px-8 pb-4 flex justify-between items-start">
      <div>
        <h3 className="text-4xl font-black text-white tracking-tighter flex items-baseline gap-3">
          {appointment.person_name}
          <span className="w-2 h-2 bg-green-500 rounded-full animate-pulse" title="Active Appointment" />
        </h3>
      </div>
      <button
        onClick={onClose}
        className="p-2 bg-slate-800/50 hover:bg-rose-500/20 hover:text-rose-500 text-slate-500 rounded-full transition-all"
      >
        <X size={20} />
      </button>
    </div>

    <div className="p-8 pt-4 space-y-8">
      <div className="flex flex-wrap gap-3">
        <div className="bg-slate-800/30 px-4 py-2 rounded-lg border-b-2 border-slate-700">
          <p className="text-[10px] font-bold text-slate-500 uppercase tracking-widest flex items-center gap-2">
            <Calendar size={10}/> Scheduled
          </p>
          <p className="text-blue-100 font-mono text-sm">{appointment.display_date}</p>
        </div>
        <div className="bg-slate-800/30 px-4 py-2 rounded-lg border-b-2 border-slate-700">
          <p className="text-[10px] font-bold text-slate-500 uppercase tracking-widest flex items-center gap-2">
            <Clock size={10}/> Window
          </p>
          <p className="text-blue-100 font-mono text-sm">{appointment.start_time} — {appointment.end_time}</p>
        </div>
      </div>

      <div className="relative group">
        <p className="text-xs font-black text-slate-500 uppercase tracking-tighter mb-2 flex items-center gap-2">
          <MapPin size={14} className="text-blue-500" /> Location
        </p>
        <div className="bg-white/[0.03] hover:bg-white/[0.05] p-5 rounded-2xl border border-white/10 transition-colors">
          <p className="text-slate-300 font-medium leading-snug">
            {appointment.address}
          </p>
        </div>
      </div>

      <div className="space-y-3 relative">
        <div className="absolute -left-8 top-0 bottom-0 w-1 bg-slate-800/50" />
        <p className="text-xs font-black text-slate-500 uppercase tracking-tighter flex items-center gap-2">
          <AlignLeft size={14} className="text-blue-500" /> Briefing
        </p>
      </div>
    </div>
  </div>

```

```

    <div className="text-slate-400 leading-relaxed text-sm font-medium selection:bg-bl
e-500 selection:text-white">
      {appointment.details ? (
        <div className="space-y-2">
          {appointment.details.split('
').map((line, i) => (
            <p key={i} className="border-b border-white/5 pb-1">{line}</p>
          ))}
        </div>
      ) : (
        <span className="italic opacity-30">No special instructions provided...</span>
      )}
    </div>
  </div>
</div>

  <div className="p-8 bg-white/[0.02] flex justify-between items-center">
    <div className="hidden sm:block">
      <p className="text-[9px] text-slate-600 font-mono uppercase">Appointment ID: {appoi
ntment.id || 'N/A'}</p>
    </div>
    <button
      onClick={onClose}
      className="px-10 py-4 bg-white text-black hover:bg-blue-600 hover:text-white rounde
d-xl font-black text-xs uppercase tracking-[0.2em] transition-all shadow-lg active:scale-95"
    >
      Close Record
    </button>
  </div>
</div>
</div>
);
}

export function AppointmentBooking({ availability, jobDuration, onConfirm, onCancel, price }) {
  const [selectedDate, setSelectedDate] = useState('');
  const [selectedTime, setSelectedTime] = useState('');
  const [formData, setFormData] = useState({
    address: '',
    details: ''
  });

  const parseDurationToHours = (durationStr) => {
    if (!durationStr || !durationStr.includes(':')) return 0;
    const [hours, mins] = durationStr.split(':').map(Number);
    return hours + (mins / 60);
  };

  const calculateEndTime = (startTime, durationStr) => {
    if (!startTime || !durationStr) return '';

    const [startH, startM] = startTime.split(':').map(Number);
    const [durH, durM] = durationStr.split(':').map(Number);

    const totalMinutes = (startH * 60) + startM + (durH * 60) + durM;

    const endH = Math.floor(totalMinutes / 60) % 24;
    const endM = totalMinutes % 60;

    return `${endH.toString().padStart(2, '0')}:${endM.toString().padStart(2, '0')}`;
  };
}

```

```

const availableDates = Object.keys(availability);
const timeSlots = selectedDate ? availability[selectedDate] : [];

const handleConfirm = () => {
  if (!selectedDate || !selectedTime || !formData.address) {
    alert("Please fill in all required fields");
    return;
  }

  const endTime = calculateEndTime(selectedTime, jobDuration);

  onConfirm({
    date: selectedDate,
    start_time: selectedTime,
    end_time: endTime,
    address: formData.address,
    details: formData.details,
  });
};

return (
  <div className="bg-slate-800/60 border border-slate-700 p-8 rounded-3xl shadow-2xlrbackd
op-blur-xl animate-fadeIn space-y-8 max-w-2xl mx-auto">
    <h2 className="text-2xl font-bold text-white flex items-center gap-3">
      <CalendarIcon className="text-blue-500" /> Book Appointment
    </h2>

    <div className="space-y-4">
      <label className="text-slate-400 text-sm font-bold uppercase tracking-wider blockt
ext-center mb-2">
        Select Date
      </label>

      <div className="grid grid-cols-4 sm:grid-cols-7 gap-2">
        {availableDates.map(date => (
          <button
            key={date}
            onClick={() => { setSelectedDate(date); setSelectedTime('
            '); }}
            className={`py-3 px-1 rounded-xl border text-sm font-medium transition-
all ${
              selectedDate === date
                ? 'bg-blue-600 border-blue-400 text-white shadow-lg shadow-blue-50
0/20'
                : 'bg-slate-900/40 border-slate-700 text-slate-300 hover:border-sl
ate-500'
            }}
          >
            <span className="block text-[10px] uppercase opacity-60">
              {new Date(date).toLocaleDateString(undefined, { month: 'short' })}
            </span>
            <span className="text-base">
              {new Date(date).toLocaleDateString(undefined, { day: 'numeric' })}
            </span>
          </button>
        ))}
      </div>
    </div>

    {selectedDate && (
      <div className="space-y-4 animate-fadeIn">
        <label className="text-slate-400 text-sm font-bold uppercase tracking-wider f

```



```

border border-blue-500/20">
    {parseDurationToHours(jobDuration)} hour Service
  </span>
</div>

<div className="space-y-4">
  <div className="flex items-center gap-3">
    <div className="p-2 bg-slate-800 rounded-lg">
      <Clock className="w-5 h-5 text-blue-400" />
    </div>
    <div>
      <p className="text-white font-bold text-lg">
        {selectedTime} — {calculateEndTime(selectedTime, jobDuration)}
      </p>
      <p className="text-xs text-slate-500">
        {new Date(selectedDate).toLocaleDateString(undefined, {
          weekday: 'long',
            month: 'short',
            day: 'numeric'
          })}
      </p>
    </div>
  </div>

  <div className="flex items-center gap-3">
    <div className="p-2 bg-slate-800 rounded-lg">
      <MapPin className="w-5 h-5 text-red-400" />
    </div>
    <div>
      <p className="text-white font-medium text-sm leading-tight">
        {formData.address}
      </p>
      <p className="text-[10px] text-slate-500 uppercase tracking-tighter">
        Service Location
      </p>
    </div>
  </div>

  <div className="flex items-center gap-3">
    <div className="p-2 bg-slate-800 rounded-lg">
      <DollarSign className="w-5 h-5 text-green-400" />
    </div>
    <div>
      <p className="text-green-400 font-bold text-lg">
        ${((parseFloat(price) * parseDurationToHours(jobDuration)).toFixed(
2))}
      </p>
      <p className="text-[10px] text-slate-500 uppercase tracking-tighter">
        Total Estimate (${price}/hr × {parseDurationToHours(jobDuration)}
hrs)
      </p>
    </div>
  </div>
</div>
)}

<div className="space-y-4">
  <div className="flex items-start gap-3 p-4 bg-blue-500/5 border border-blue-500/2
0 rounded-2xl">
    <CheckCircle className="w-4 h-4 text-blue-400 mt-1" />
    <p className="text-slate-400 text-sm leading-relaxed">

```

```

        By confirming, you agree to the hourly fee of <span className="text-white font-bold">${price}</span>.
        <br/>By confirming you also agree to knowledge that<br/>Pro-Find is not
        accountable to any price changes the freelancer might impose
    </p>
</div>

<div className="flex gap-4">
    <button
        onClick={onCancel}
        className="flex-1 py-4 border border-slate-700 text-slate-400 font-bold rounded-2xl hover:bg-slate-700/30 transition-all"
    >
        Cancel
    </button>

    <button
        disabled={!selectedDate || !selectedTime || !formData.address}
        onClick={handleConfirm}
        className={`flex-[2] py-4 font-bold rounded-2xl shadow-lg flex items-center justify-center gap-2 transition-all
            ${(!selectedDate || !selectedTime || !formData.address)
                ? "bg-slate-700 text-slate-500 cursor-not-allowed opacity-50"
                : "bg-gradient-to-r from-green-600 to-emerald-600 hover:from-green-500 hover:to-emerald-500 text-white transform hover:-translate-y-1"}
        `}
    >
        <CheckCircle className="w-5 h-5" /> Confirm Booking
    </button>
</div>
</div>
</div>
);
}

```

```
import { useEffect } from "react";
import { useLocation } from "react-router-dom";

export default function RouteTracker(){
  const location = useLocation()

  useEffect(() => {
    localStorage.setItem("lastRoute", location.pathname)
  }, [location,])
  return null
}
```



```
export default function MsgBuild(type, payload, token, type_of_data="JSON"){
  const my_dict = {
    "type": type,
    "data": payload,
    "data_type": type_of_data,
    "token": token
  }
  return JSON.stringify(my_dict)
}
```

```
//Messages are sent in this format:
//{
//  type: type of message...
//  data: actual payload of the message, if more then one field then sent as a dict
//  data_type: BYTES or JSON
//}

export default function websocketParser(payload)
{
  try{
    const msgObj = JSON.parse(payload);
    const {type, data, data_type, token} = msgObj;
    let parsedData = data
    if (data_type == "BYTES"){
      const binaryStr = atob(data);
      const bytes = new Uint8Array(binaryStr.length);
      for (let i=0; i<binaryStr.length; i++){
        bytes[i] = binaryStr.charCodeAt(i);
      }
      parsedData = bytes;
    }

    return {type, data:parsedData, token};
  } catch(err){
    console.error("Failed to parse message", err);
    return null;
  }
}
```

```
export const MessageTypes = Object.freeze({
  //Anything
  BROAD: "BRD",

  //Authentication
  LOGIN: "LOGIN",
  USER_SIGNUP: "USERSIGNUP",
  FREELANCER_SIGNUP: "FRLNCSIGNUP",
  VERIFICATION: "VERIFYING",
  FORGOT_PASSWORD_REQUEST: "FRGT_PAS_RQT",
  FORGOT_PASSWORD_AUTHENTICATION: "FRGT_PASS_AUTH",
  CHANGE_PASS: "CHNG_PAS",

  //Homepage
  GET_USER_INFO: "USR_INFO",
  UPDATE_APPOINTMENTS_STATUS: "UPDT_APP_STAT",
  MARK_READ_NOTIFICATION: "RD_NTFC",
  GET_APPOINTMENT_TIMES: "GET_APP_TIME",
  MAKE_APPOINTMENT: "MK_APP",

  //Searchg
  GET_PUBLIC_PROFILE_INFO: "PBL_INFO",
  GET_MINIMAL_FREELANCER_INFO: "MNML_VIEW",
  GET_JOBS: "GET_JBS",
  GET_CITIES_BY_JOB: "GET_CITIS"
});

export const StatusMessage = Object.freeze({
  //Anything
  TOKEN_BAD: "BAD_TKN",

  //Authentication
  LOGGED_IN: "LOGGED",
  FAILED_LOG_IN: "FAILED_LOG",

  SIGNING_UP: "SIGNED",
  FAILED_SIGN_UP: "FAILED_SIGNED",

  VERIFICATION_GOOD: "VERI_GOOD",
  VERIFICATION_BAD: "VERI_BAD",

  FORGOT_PASSWORD_GOOD: "FRGT_GOOD",
  FORGOT_PASSWORD_BAD: "FRGT_BAD",

  CHANGE_PASSWORD_GOOD: "CHNG_GOOD",
  CHANGE_PASSWORD_BAD: "CHNG_BAD",

  //Homepage
  GOT_USER_INFO: "GOT_USR_INFO",
  FAILED_TO_GET_USER_INFO: "FLD_USR_INFO",

  UPDATED_APP_STATUS: "UPDT_APP",
  FAILED_TO_UPDATE_APP_STATUS: "FAILED_UPDT_APP",

  MARKED_READ_NOTIFICATIONS: "READ_NOTIFICATIONS",
  FAILED_TO_MARK_READ_NOTIFICATIONS: "FAILED_READ_NOTIFICATIONS",

  GOT_APPOINTMENT_TIMES: "GOT_APP_TIME",
  FAILED_TO_GET_APPOINTMENT_TIMES: "FLD_APP_TIME",
```

```
    BOOKED_APPOINTMENT: "BKD_APP",
    FAILED_TO_BOOK_APPOINTMENT: "FLD_BKD_APP",

    //Search
    GOT_JOBS: "GOT_JBS",
    FAILED_TO_GET_JOBS: "FLD_JBS",

    GOT_CITIES: "GOT_CITIS",
    FAILED_TO_GET_CITIES: "FLD_CITIS",

    GOT_MINIMAL_INFO: "GOT_MNML",
    FAILED_TO_GET_MINIMAL_INFO: "FLD_MNML",

    GOT_FREELANCER_PUBLIC_INFO: "GOT_FREE",
    FAILED_TO_GET_FREELANCER_PUBLIC_INFO: "FLD_FREE",
  })
```

```

import MsgBuild from "../MsgBuilder"
import {useSocket} from '../socket/SocketContext'
import { MessageTypes } from "../MsgTypes"

//#region Login
export function LoginRequest(username, password, ws, token=""){
  const payload = {
    "email": username,
    "password": password
  }
  const msg = MsgBuild(MessageTypes.LOGIN, payload, token, "JSON")
  console.log(`Sending - ${msg}`)
  ws.send(msg)
}
//#endregion

//#region SignUp
export function SignUpRequest(ws, account_info, type_of_user = "User", token=""){
  let payload = account_info;
  payload.role = type_of_user;
  let msg;
  if (type_of_user === "User"){
    msg = MsgBuild(MessageTypes.USER_SIGNUP, payload, token, "JSON")
  }
  else{
    msg = MsgBuild(MessageTypes.FREELANCER_SIGNUP, payload, token, "JSON")
  }
  console.log(`Sending - ${msg}`)
  ws.send(msg)
}
//#endregion

//#region Verification
export function VerificationRequest(ws, email, veri_code, role, token=""){
  let payload = {"email":email, "verification_code":veri_code, "role":role}
  let msg = MsgBuild(MessageTypes.VERIFICATION, payload, token, "JSON")
  ws.send(msg)
}
//#endregion

//#region Forgot Password
export function ForgotPasswordVerificationCodeRequest(ws, email, token=""){
  let payload = {"email":email}
  let msg = MsgBuild(MessageTypes.FORGOT_PASSWORD_REQUEST, payload, token, "JSON")
  ws.send(msg)
}

export function ForgotPasswordAuthenticationRequest(ws, email, code, token=""){
  let payload = {"email":email, "veri_code":code}
  let msg = MsgBuild(MessageTypes.FORGOT_PASSWORD_AUTHENTICATION, payload, token, "JSON")
  ws.send(msg)
}

export function ChangePasswordRequest(ws, email, pass, token=""){
  let payload = {"email":email, "password":pass }
  let msg = MsgBuild(MessageTypes.CHANGE_PASS, payload, token, "JSON")
  ws.send(msg)
}

```

```
}
//endregion

//region HomePage
export function UserInfoRequest(ws, token){
  const [encodedData] = token.split('.')
  const decodedJson = atob(encodedData)
  const data = JSON.parse(decodedJson)
  let payload = {"email":data.email}
  let msg = MsgBuild(MessageTypes.GET_USER_INFO, payload, token, "JSON")
  ws.send(msg)
}

export function UpdateAppointmentsStatusRequest(ws, apps, token){
  let payload = {"appointments":apps}
  let msg = MsgBuild(MessageTypes.UPDATE_APPOINTMENTS_STATUS, payload, token, "JSON")
  console.log(`About to send - ${msg}`)
  ws.send(msg)
}

export function MarkReadNotificationsRequest(ws, userId, token){
  let payload = {"user_id":userId}
  let msg = MsgBuild(MessageTypes.MARK_READ_NOTIFICATION, payload, token, "JSON")
  ws.send(msg)
}

export function GetAvailableWorkTimes(ws, targetId, token){
  let payload = {"id":targetId}
  let msg = MsgBuild(MessageTypes.GET_APPOINTMENT_TIMES, payload, token, "JSON")
  ws.send(msg)
}

export function BookAppointment(ws, app, token){
  let payload = {"app":app}
  let msg = MsgBuild(MessageTypes.MAKE_APPOINTMENT, payload, token, "JSON")
  console.log(`sending - ${msg}`)
  ws.send(msg)
}

//endregion

//region Search
export function GetPublicProfileInfoRequest(ws, targetId, token){
  let payload = {id:targetId}
  let msg = MsgBuild(MessageTypes.GET_PUBLIC_PROFILE_INFO, payload, token, "JSON")
  ws.send(msg)
}

export function GetMinimalFreelancerInfo(ws, token, city, job){
  let payload = {"city":city, "job":job}
  let msg = MsgBuild(MessageTypes.GET_MINIMAL_FREELANCER_INFO, payload, token, "JSON")
  ws.send(msg)
}

export function GetAllJobs(ws, token)
```

```
{
  let msg = MsgBuild(MessageTypes.GET_JOBS, "", token, "JSON")
  ws.send(msg)
}

export function GetCitiesByJob(ws, job, token){
  let payload = {"job":job}
  let msg = MsgBuild(MessageTypes.GET_CITIES_BY_JOB, payload, token, "JSON")
  ws.send(msg)
}

//#endregion
```

```
import {MessageTypes, StatusMessage} from "../MsgTypes"
import {login, setUserInfo} from "../store/authSlice"
import { appointmentsRecvd } from "../store/appointmentsSlice";

//#region Login
export const handleLoginResponse = (payload, dispatch) =>{
  if (MessageTypes.LOGIN == payload.type)
    if (StatusMessage.LOGGED_IN in payload.data)
    {
      dispatch(login(payload.data[StatusMessage.LOGGED_IN]))
      return [true, payload.data[StatusMessage.LOGGED_IN]];
    }
    else if (StatusMessage.FAILED_LOG_IN in payload.data)
      return [false, payload.data[StatusMessage.FAILED_LOG_IN]];
    else
      console.log("Unknown response")
  else
    console.log("Unknown message type")
}
//#endregion

//#region SignUp
export const handleSignUpResponse = (payload) =>{
  let signupFailed = StatusMessage.FAILED_SIGN_UP;
  let signupSuccess = StatusMessage.SIGNING_UP;
  if (MessageTypes.USER_SIGNUP === payload.type || MessageTypes.FREELANCER_SIGNUP === payload.type){
    if (payload.data && signupFailed in payload.data){
      console.log("Failed")
      return [false, payload.data[signupFailed]]
    }
    else{
      return [true, payload.data[signupSuccess]]
    }
  }
}
//#endregion

//#region Verification
export const handleVerificationReponse = (payload) =>{
  let data = payload.data
  if (StatusMessage.VERIFICATION_BAD in data){
    return [false, data[StatusMessage.VERIFICATION_BAD]]
  }
  else if (StatusMessage.VERIFICATION_GOOD in data)
    return [true, data[StatusMessage.VERIFICATION_GOOD]]
  return [false, "Server error"]
}
//#endregion

//#region Forgot password
export const handleForgotPasswordVerificationCodeResponse =(payload) =>{
  let data = payload.data
  if (StatusMessage.FORGOT_PASSWORD_GOOD in data)
```



```

        return [true, data[StatusMessage.FORGOT_PASSWORD_GOOD]]
      else
        return [false, data[StatusMessage.FORGOT_PASSWORD_BAD]]
    }

export const handleForgotPasswordVerifyCodeResponse = (payload) =>{
  let data = payload.data
  if (StatusMessage.FORGOT_PASSWORD_GOOD in data)
    return [true, StatusMessage.FORGOT_PASSWORD_GOOD]
  else
    return [false, StatusMessage.FORGOT_PASSWORD_BAD]
}

export const handleChangePasswordResponse = (payload) =>{
  let data = payload.data
  if (StatusMessage.CHANGE_PASSWORD_BAD in data)
    return [false, data[StatusMessage.CHANGE_PASSWORD_BAD]]
  else
    return [true, data[StatusMessage.CHANGE_PASSWORD_GOOD]]
}

//#endregion

//#region Homepage
export const handleUserInfoResponse = (dispatch, payload) =>{
  let data = payload.data

  if (StatusMessage.FAILED_TO_GET_USER_INFO in data){
    return [false, "Issue retrieving info"]
  }
  else{
    const accInfo = data[StatusMessage.GOT_USER_INFO]
    let toInsert = {"user":{}}
    for (const key in accInfo){
      if (key === "appointments" || key === "notifications") continue;

      toInsert["user"][key] = accInfo[key]
    }
    toInsert["user"]["id"] = data["user_id"]
    toInsert["notifications"] = accInfo["notifications"]
    dispatch(setUserInfo(toInsert))
    dispatch(appointmentsRecvd(accInfo["appointments"]))
    return [true, data["user_id"]]
  }
}

export const handleUpdatedAppointmentsResponse = (payload) =>{
  let data = payload.data
  if (StatusMessage.FAILED_TO_UPDATE_APP_STATUS in data)
    return [false, "Failed to update status"]
  else if (StatusMessage.UPDATED_APP_STATUS in data)
    return [true, ""]
}

export const handleMarkedReadNotifications = (payload) =>{
  let data = payload.data
  if (StatusMessage.MARKED_READ_NOTIFICATIONS in data){
    return [true, ""]
  }
}

```

```
    }
    else{
        return [false, data[StatusMessage.FAILED_TO_MARK_READ_NOTIFICATIONS]]
    }
}

export const handleAppointmentTimes = (payload) =>{
    let data = payload.data
    if (StatusMessage.GOT_APPOINTMENT_TIMES in data)
        return [true, data[StatusMessage.GOT_APPOINTMENT_TIMES]]
    else
        return [false, data[StatusMessage.FAILED_TO_GET_APPOINTMENT_TIMES]]
}

export const handleAppointmentMadeResponse = (payload) =>{
    let data = payload.data

    if (StatusMessage.BOOKED_APPOINTMENT in data)
        return [true, ""]
    else
        return [false, data[StatusMessage.FAILED_TO_BOOK_APPOINTMENT]]
}

//#endregion

//#region Search
export const handleProfileInfoResponse = (payload) => {
    let data = payload.data
    if (StatusMessage.GOT_FREELANCER_PUBLIC_INFO in data){
        return [true, data[StatusMessage.GOT_FREELANCER_PUBLIC_INFO]]
    }
    else{
        return [false, data[StatusMessage.FAILED_TO_GET_FREELANCER_PUBLIC_INFO]]
    }
}

export const handleJobsResponse = (payload) =>{
    let data = payload.data
    console.log(data)
    if (StatusMessage.GOT_JOBS in data){
        return [true, data[StatusMessage.GOT_JOBS]["jobs"]]
    }
    else{
        return [false, data[StatusMessage.FAILED_TO_GET_JOBS]]
    }
}

export const handleCitiesResponse = (payload) =>{
    let data = payload.data
    if (StatusMessage.GOT_CITIES in data)
        return [true, data[StatusMessage.GOT_CITIES]["cities"]]
    else
        return [false, data[StatusMessage.FAILED_TO_GET_CITIES]]
}
```

```
export const handleMinimalFreelancerInfoResponse = (payload) =>{
  let data = payload.data
  if (StatusMessage.GOT_MINIMAL_INFO in data){
    return [true, data[StatusMessage.GOT_MINIMAL_INFO]]
  }
  else
    return [false, data[StatusMessage.FAILED_TO_GET_MINIMAL_INFO]]
}

//#endregion
```

```
import {createContext, useContext, useState, useEffect} from 'react'

const SocketContext = createContext(null)
let wsSingleton = null;

const getWebSocket = (ip, port) =>{
  if (!wsSingleton){
    wsSingleton = new WebSocket(`wss://${ip}:${port}`);
    wsSingleton.onopen =() =>console.log("Socket Opened");
    wsSingleton.onclose=() =>console.log("Socket Closed");
  }
  return wsSingleton;
}

export const SocketProvider = ({children}) =>{
  const [socket, setSocket] = useState(null)

  useEffect(() =>{
    const IP = "omer.ystr";
    const PORT = "1111";
    const ws = getWebSocket(IP, PORT)
    setSocket(ws)
  }, [])

  return (
    <SocketContext.Provider value={socket}>
      {children}
    </SocketContext.Provider>
  )
}

export const useSocket = () => useContext(SocketContext)
```

```
import { createSlice } from '@reduxjs/toolkit'

const initialState = {
  userAppointments: [],
  freelancerAppointments: []
}

const appointmentSlice = createSlice({
  name: "appointments",
  initialState,
  reducers: {
    appointmentsRecvd(state, action) {
      state.userAppointments = action.payload
      state.freelancerAppointments = action.payload
    },
    appointmentAddedForUser(state, action) {
      state.userAppointments.push(action.payload)
    },
    appointmentAddedForFreelancer(state, action) {
      state.freelancerAppointments.push(action.payload)
    },
    appointmentRemovedForUser(state, action) {
      state.userAppointments = state.userAppointments.filter(a => a !== action
        .payload)
    },
    appointmentRemovedForFreelancer(state, action) {
      state.freelancerAppointments = state.freelancerAppointments.filter(a => a !== action
        .payload)
    }
  }
})

export const {
  appointmentsRecvd,
  appointmentAddedForUser,
  appointmentAddedForFreelancer,
  appointmentRemovedForUser,
  appointmentRemovedForFreelancer
} = appointmentSlice.actions

export default appointmentSlice.reducer
```

```
import {createSlice} from '@reduxjs/toolkit'

const initialState = {
  userToken: "",
  userInfo: {},
  loggedIn: false,
  notifications: []
}

const authSlice = createSlice({
  name: "auth",
  initialState,
  reducers:{
    login(state, info){
      state.loggedIn = true;
      state.userToken = info.payload;
    },
    setUserInfo(state, info){
      state.userInfo = info.payload.user
      state.notifications = info.payload.notifications
    },
    logout(state){
      state.userInfo = {};
      state.userToken = "";
      state.loggedIn=false;
      state.notifications=[];
    }
  }
})

export const {login, setUserInfo, logout} = authSlice.actions
export default authSlice.reducer
```

```
import { configureStore, combineReducers } from "@reduxjs/toolkit";
import { persistStore, persistReducer, FLUSH, REHYDRATE, PAUSE, PERSIST, PURGE, REGISTER } from "redux-persist";
import storage from "redux-persist/lib/storage"; // localStorage

import authReducer from './authSlice';
import appointmentReducer from './appointmentsSlice';

const rootReducer = combineReducers({
  auth: authReducer,
  appointment: appointmentReducer,
});

const persistConfig = {
  key: "root",
  storage,
};

const persistedReducer = persistReducer(persistConfig, rootReducer);

export const store = configureStore({
  reducer: persistedReducer,
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware({
      serializableCheck: {
        ignoredActions: [FLUSH, REHYDRATE, PAUSE, PERSIST, PURGE, REGISTER],
      },
    }),
});

export const persistor = persistStore(store);

export default store;
```